

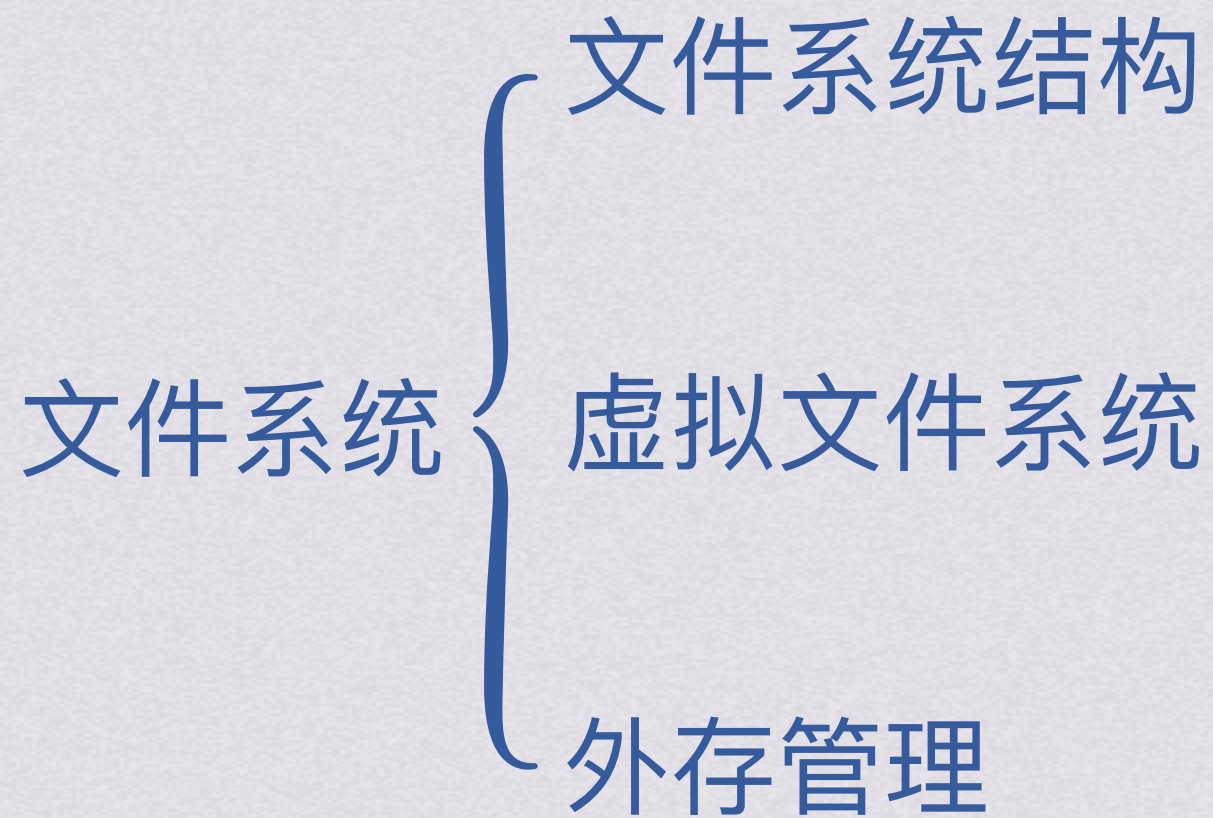
bok25

**基
礎**

課、操作系統



文件系统

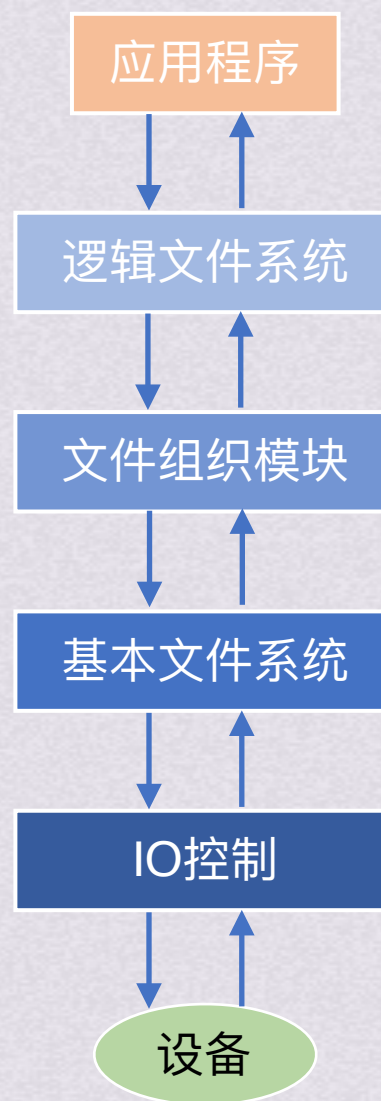




文件系统结构

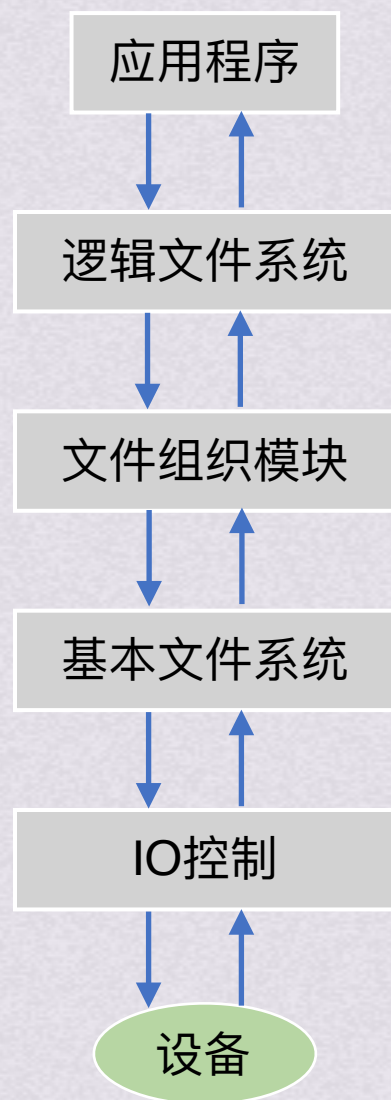


文件系统结构





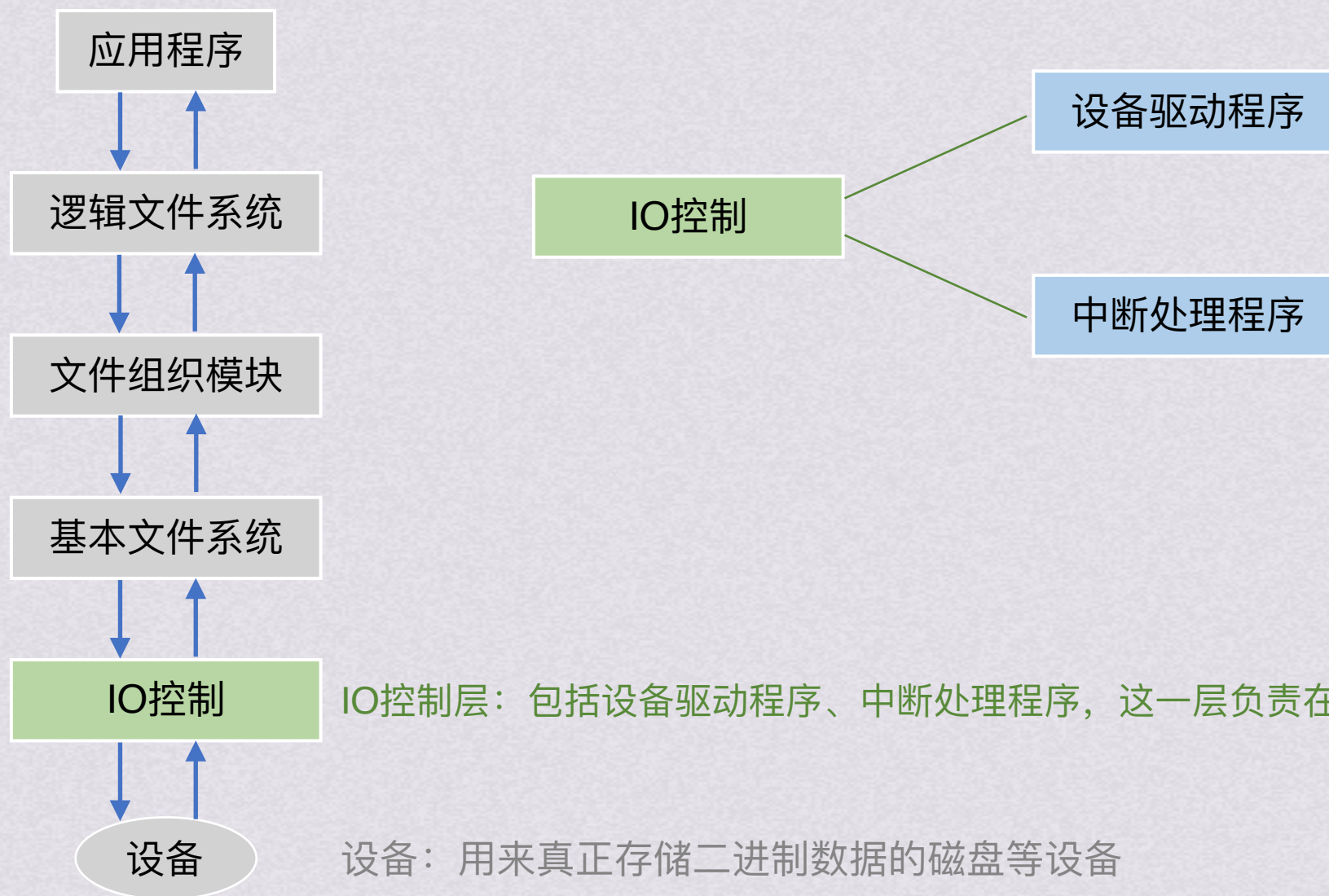
文件系统结构



设备：用来真正存储二进制数据的磁盘等设备



文件系统结构

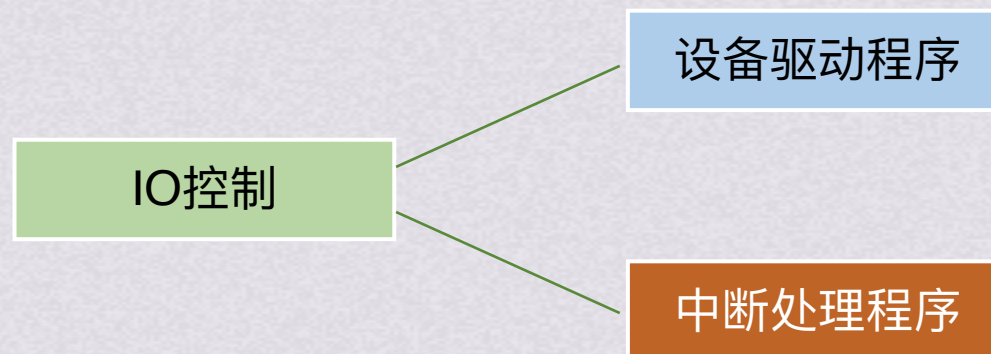
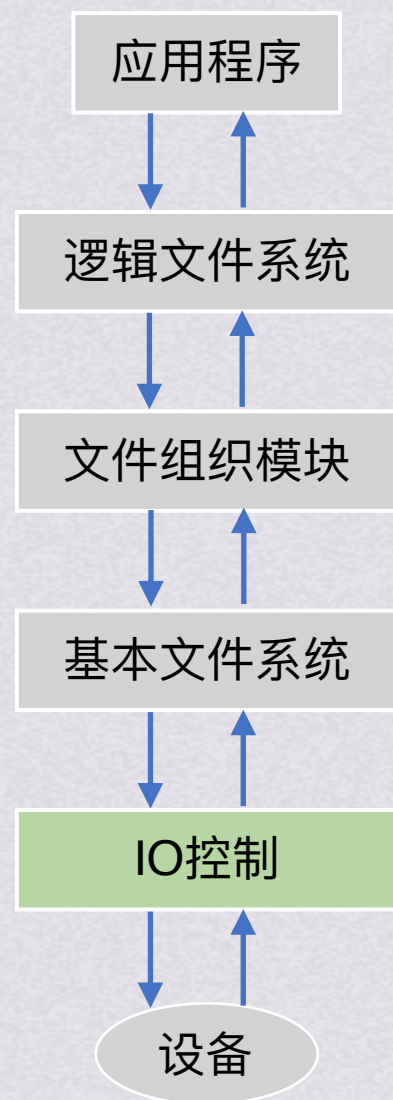


IO控制层：包括设备驱动程序、中断处理程序，这一层负责在内存和磁盘系统之间传输信息

设备：用来真正存储二进制数据的磁盘等设备



文件系统结构



之前已经进行过详细的了解了，内外存交互中不可缺少的重要部分，这里跳过不谈

IO控制层：包括设备驱动程序、中断处理程序，这一层负责在内存和磁盘系统之间传输信息

设备：用来真正存储二进制数据的磁盘等设备

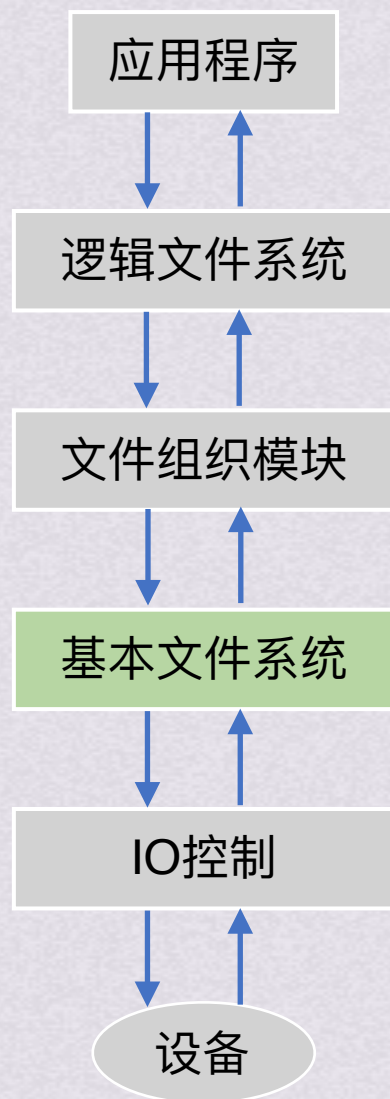


文件系统结构





文件系统结构



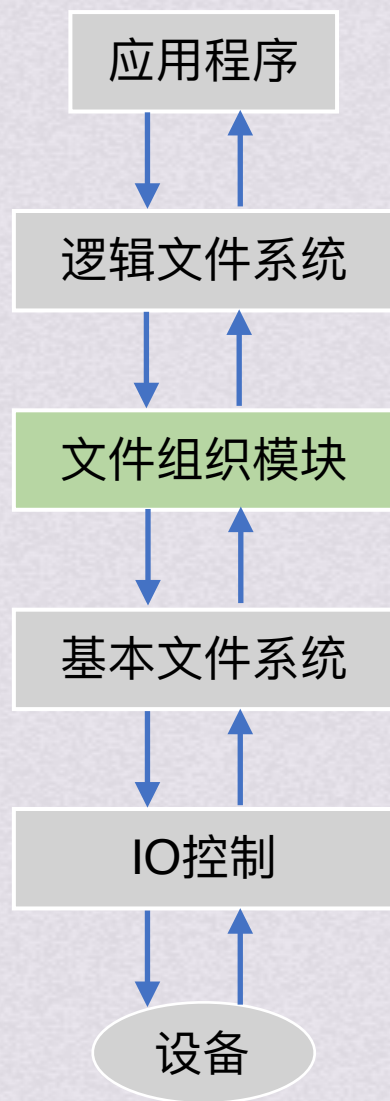
基本文件系统：向对应的设备驱动程序发送通用命令，以读取和写入磁盘的物理块。每个物理块由磁盘地址标识。同时这一层也管理内存缓冲区和各种文件系统、目录的缓存。

IO控制层：包括设备驱动程序、中断处理程序，这一层负责在内存和磁盘系统之间传输信息

设备：用来真正存储二进制数据的磁盘等设备



文件系统结构



组织文件及其逻辑块、物理块。文件组织模块可以把文件的逻辑块地址转换为物理块地址。这一层还要负责管理空闲空间

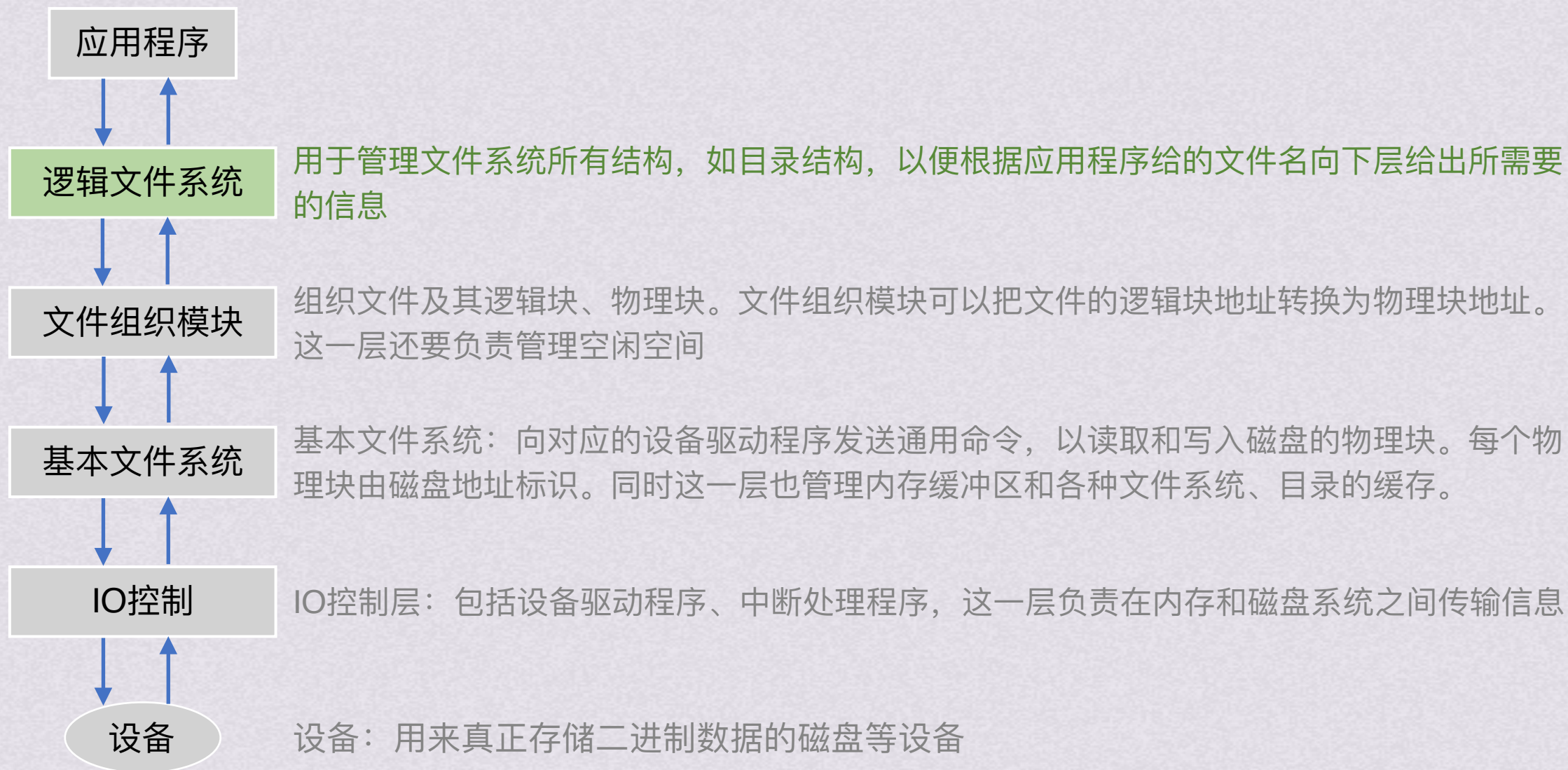
基本文件系统：向对应的设备驱动程序发送通用命令，以读取和写入磁盘的物理块。每个物理块由磁盘地址标识。同时这一层也管理内存缓冲区和各种文件系统、目录的缓存。

IO控制层：包括设备驱动程序、中断处理程序，这一层负责在内存和磁盘系统之间传输信息

设备：用来真正存储二进制数据的磁盘等设备

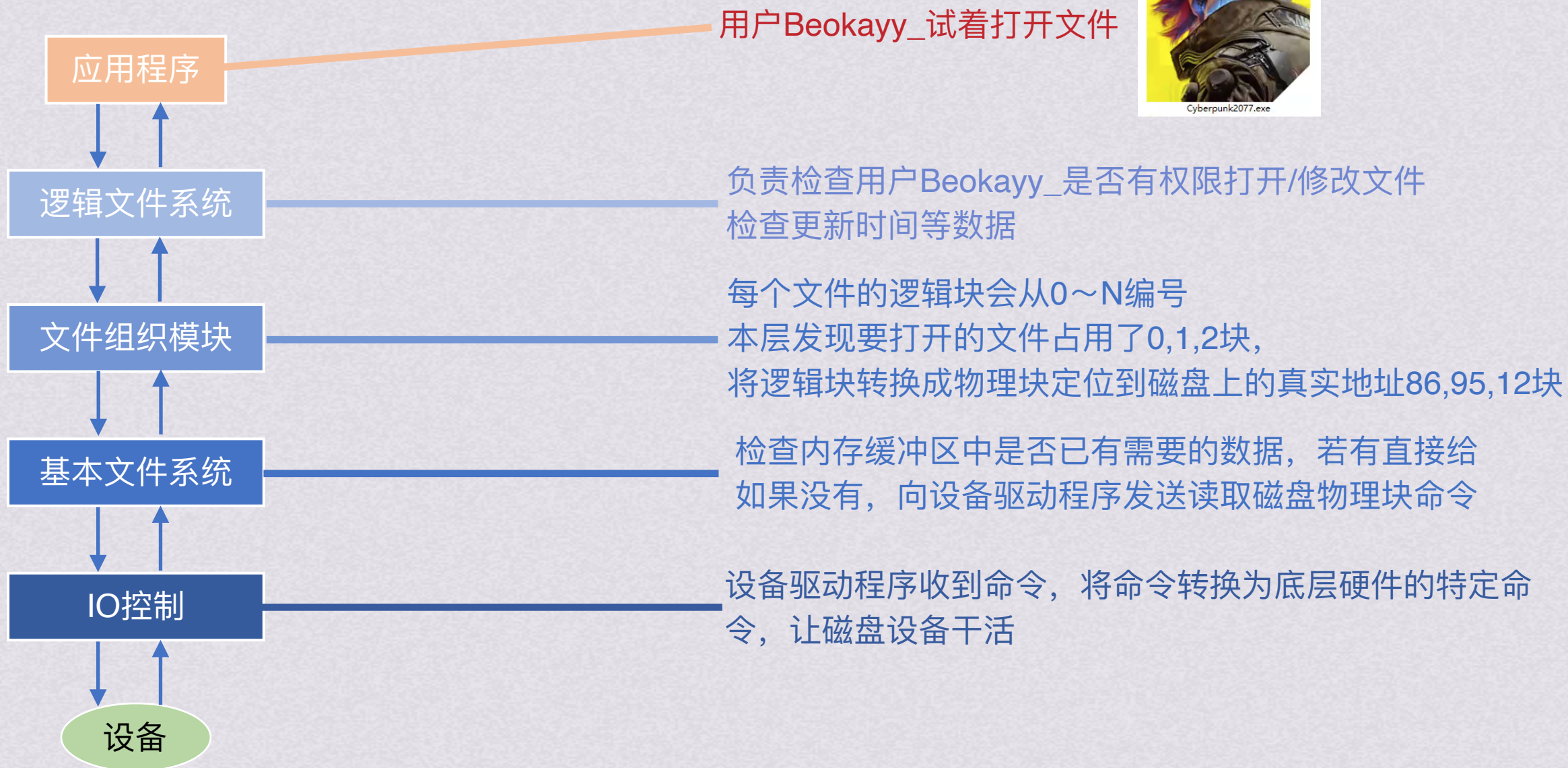


文件系统结构



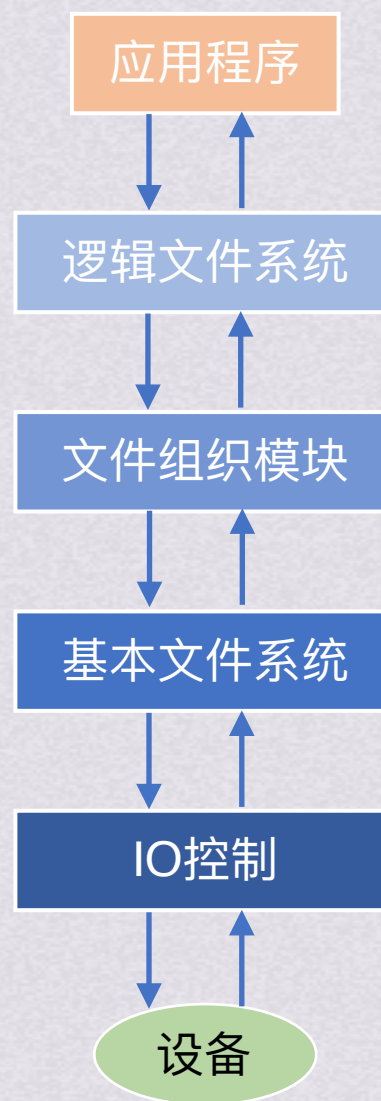


文件系统结构



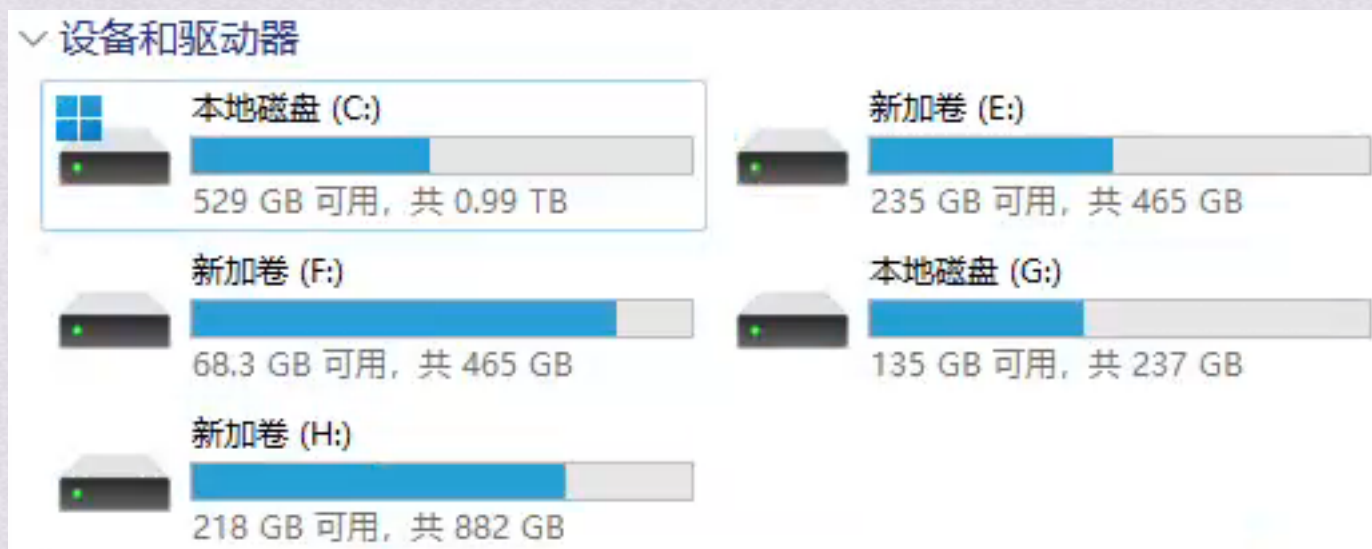


文件系统结构





文件系统结构



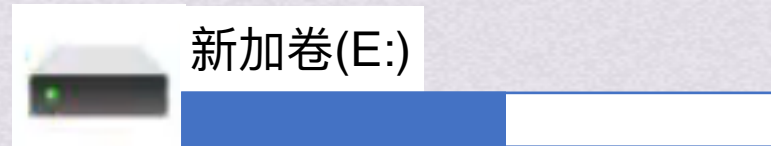
文件系统放在磁盘上，磁盘会被划分成一个或几个分区，每个分区中有一个独立的文件系统
图中的电脑共有5个独立的文件系统

文件系统中一般会存着以下信息：

- 1.启动本文件系统中操作系统的方式(如果本文件系统中操作系统)
- 2.总的块数量
- 3.空闲块的数量和位置
- 4.目录结构、各个具体的文件等

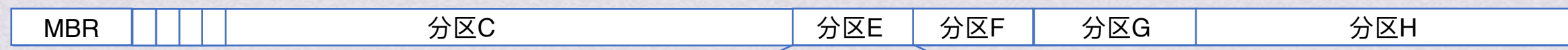


文件系统结构



文件系统结构

整个磁盘

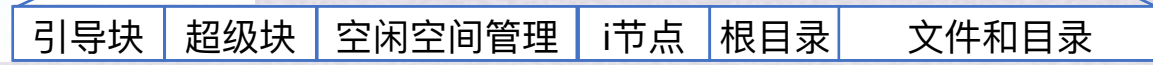


分区表

单个分区



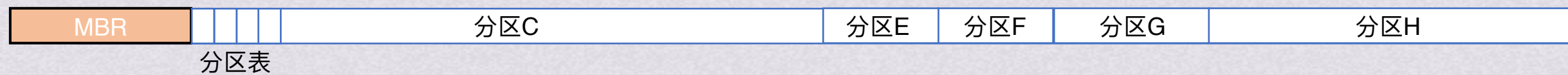
新加卷(E:)



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

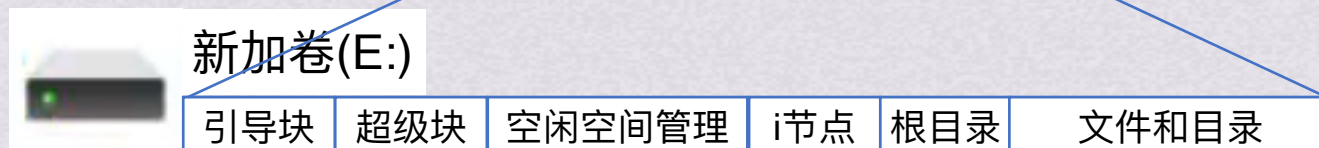
文件系统结构

整个磁盘



MBR: 主引导记录(Master Boot Record), 位于磁盘的0号扇区, 用于引导计算机。
计算机启动时, BIOS会读入并执行MBR。MBR会确定活动分区, 读入分区的第一块(引导块)

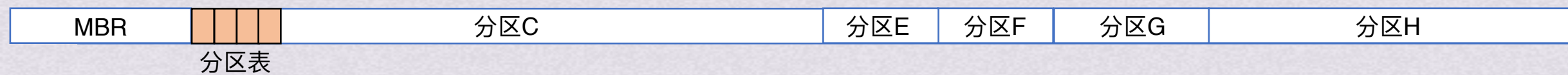
单个分区



注: 此处仅为理论上可以工作的文件系统布局, 实际上windows的文件系统并不是这种结构

文件系统结构

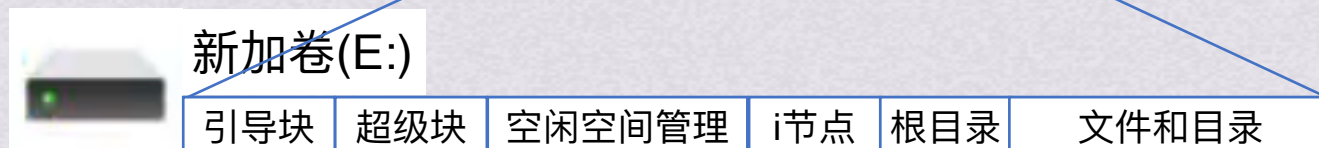
整个磁盘



MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。
计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)

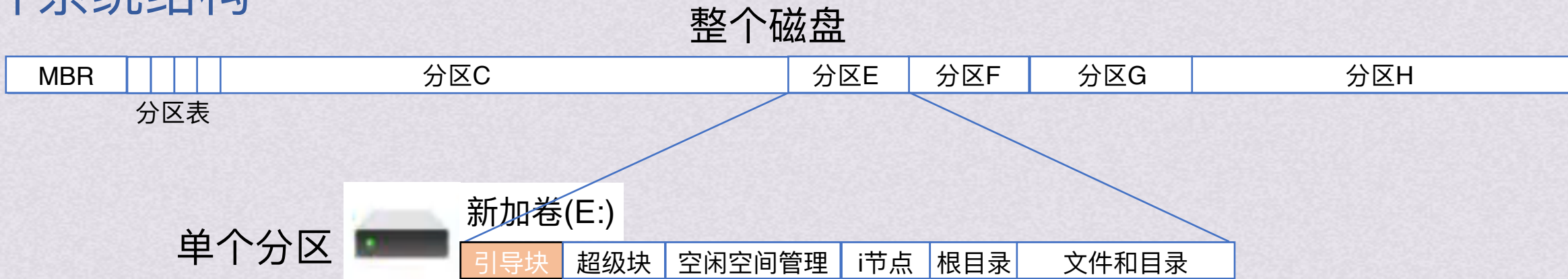
分区表：记录了每个分区的起始和结束地址。

单个分区



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

文件系统结构



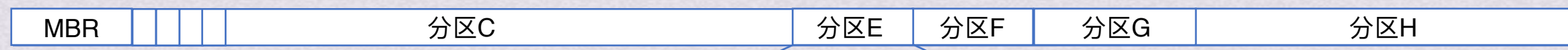
注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

引导块：MBR执行引导块中程序，该程序会启动本分区的操作系统。实际上根据不同文件系统和是否有操作系统，可能没有引导块

MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。
计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)
分区表：记录了每个分区的起始和结束地址。表中的一个分区被标记为活动分区，这样MBR就知道该去哪里读入引导块

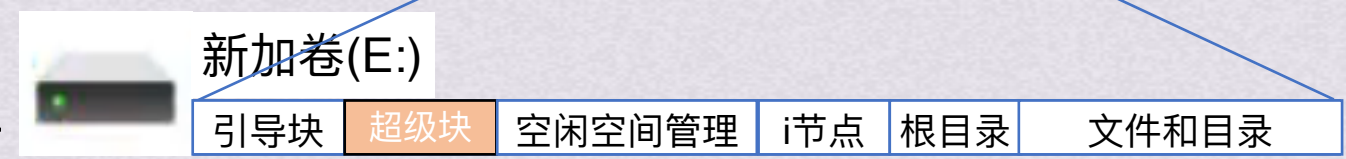
文件系统结构

整个磁盘



分区表

单个分区



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

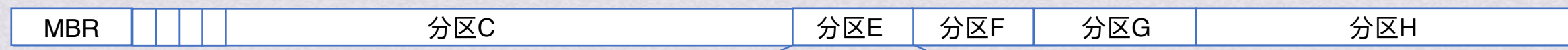
引导块：MBR执行引导块中程序，该程序会启动本分区的操作系统。实际上根据不同文件系统和是否有操作系统，可能没有引导块

超级块：包含文件系统所有关键信息，计算机启动或者该文件系统首次使用时，超级块被读入内存。一般超级块包含分区的块数量、块大小、空闲块数量和指针、空闲FCB数量和FCB指针等

MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。
计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)
分区表：记录了每个分区的起始和结束地址。表中的一个分区被标记为活动分区，这样MBR就知道该去哪里读入引导块

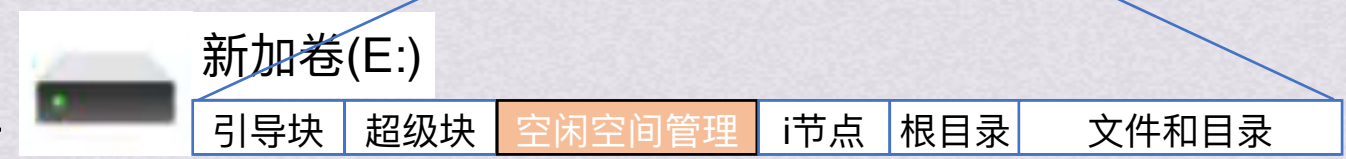
文件系统结构

整个磁盘



分区表

单个分区



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

引导块：MBR执行引导块中程序，该程序会启动本分区的操作系统。实际上根据不同文件系统和是否有操作系统，可能没有引导块

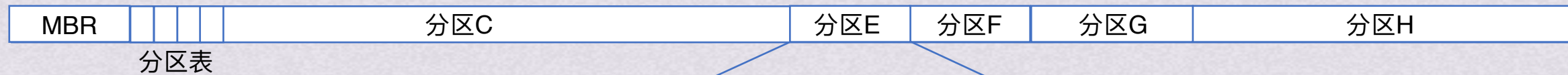
超级块：包含文件系统所有关键信息，计算机启动或者该文件系统首次使用时，超级块被读入内存。一般超级块包含分区的块数量、块大小、空闲块数量和指针、空闲FCB数量和FCB指针等

空闲空间管理：用数据结构记录并且管理分区中的空闲空间，如位示图等(马上就会学)

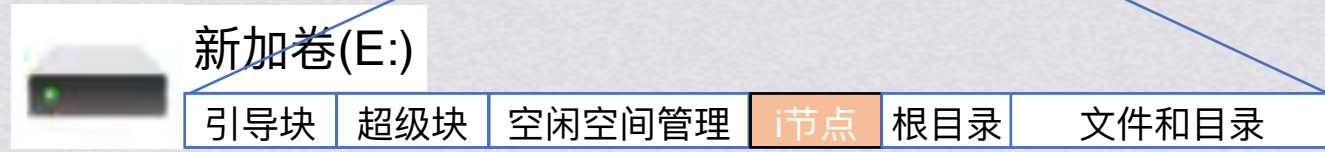
- MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。
- 计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)
- 分区表：记录了每个分区的起始和结束地址。表中的一个分区被标记为活动分区，这样MBR就知道该去哪里读入引导块

文件系统结构

整个磁盘



单个分区



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

引导块：MBR执行引导块中程序，该程序会启动本分区的操作系统。实际上根据不同文件系统和是否有操作系统，可能没有引导块

超级块：包含文件系统所有关键信息，计算机启动或者该文件系统首次使用时，超级块被读入内存。一般超级块包含分区的块数量、块大小、空闲块数量和指针、空闲FCB数量和FCB指针等

空闲空间管理：用数据结构记录并且管理分区中的空闲空间，如位示图等(马上就会学)

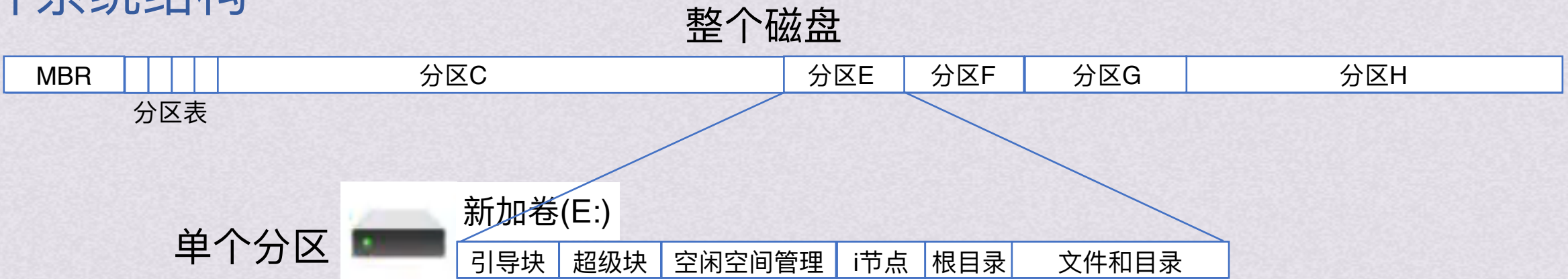
i节点：每个文件对应一个i节点，i节点描述了文件的各种信息(FCB中除了文件名的那些信息)，inode的数量限制了文件的个数

MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。

计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)

分区表：记录了每个分区的起始和结束地址。表中的一个分区被标记为活动分区，这样MBR就知道该去哪里读入引导块

文件系统结构



注：此处仅为理论上可以工作的文件系统布局，实际上windows的文件系统并不是这种结构

引导块：MBR执行引导块中程序，该程序会启动本分区的操作系统。实际上根据不同文件系统和是否有操作系统，可能没有引导块

超级块：包含文件系统所有关键信息，计算机启动或者该文件系统首次使用时，超级块被读入内存。一般超级块包含分区的块数量、块大小、空闲块数量和指针、空闲FCB数量和FCB指针等

空闲空间管理：用数据结构记录并且管理分区中的空闲空间，如位示图等(马上就会学)

i节点：每个文件对应一个i节点，i节点描述了文件的各种信息(FCB中除了文件名的那些信息)，inode的数量限制了文件的个数

MBR：主引导记录(Master Boot Record)，位于磁盘的0号扇区，用于引导计算机。
计算机启动时，BIOS会读入并执行MBR。MBR会确定活动分区，读入分区的第一块(引导块)

分区表：记录了每个分区的起始和结束地址。表中的一个分区被标记为活动分区，这样MBR就知道该去哪里读入引导块



文件系统结构

文件系统也会存放一些数据在内存中，来提高文件系统的性能表现

安装表： 包含每个已安装文件系统分区的有关信息

目录结构缓存： 包含了最近访问目录的信息

整个系统的打开文件表： 包含每个打开文件的FCB副本、打开计数和其他信息

每个进程的打开文件表： 包含进程打开文件的文件描述符(windows下叫文件句柄，Linux中称为文件描述符)和指向系统打开文件表中对应表项的指针



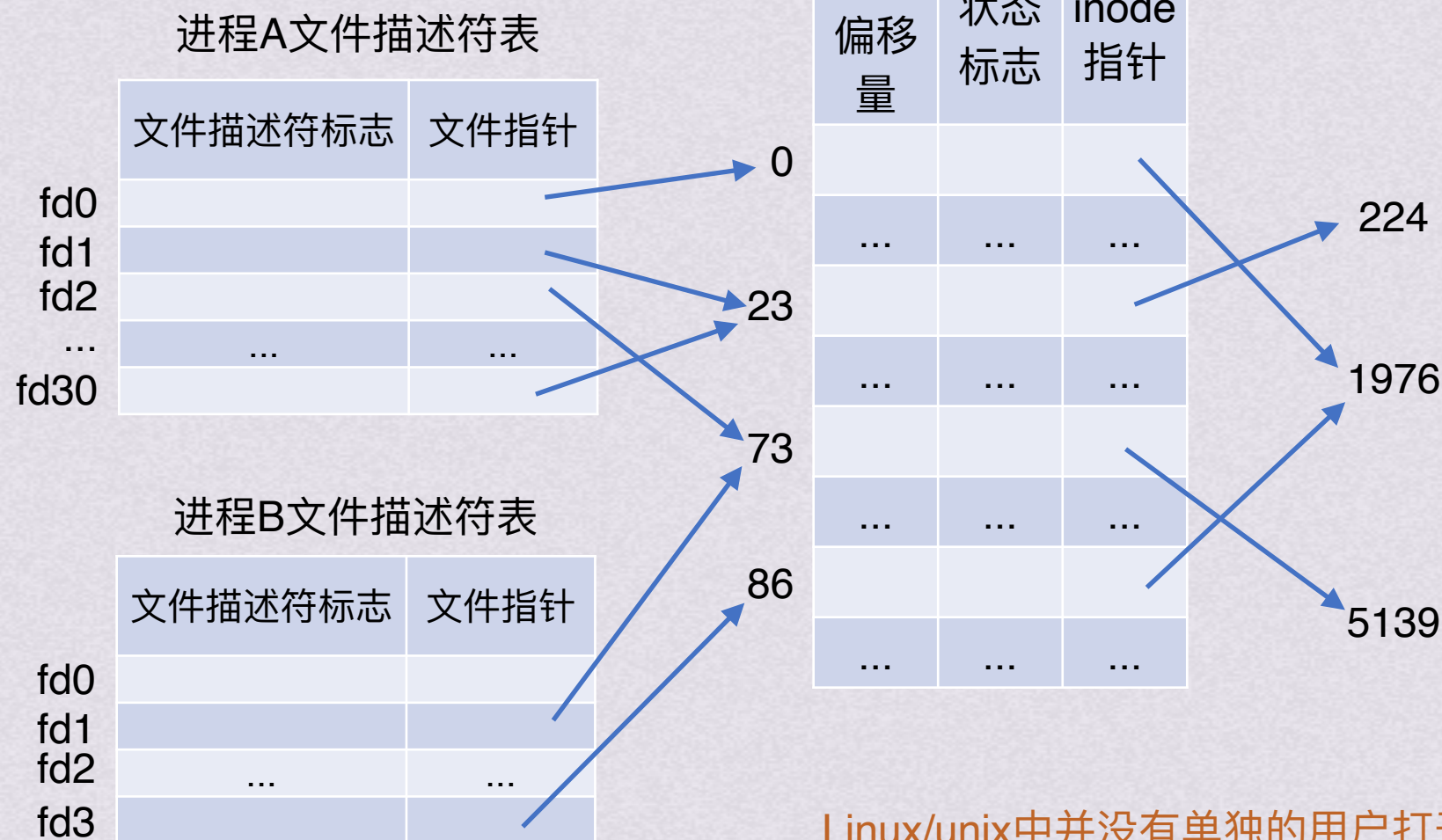
内存

文件系统结构

文件描述符、打开文件表和inode之间的关系

打开文件表(系统级)

inode表(系统级)



Linux/unix中并没有单独的用户打开文件表!



文件系统结构

练一练

[2017]若文件f1的硬链接为f2，两个进程分别打开f1和f2，获得对应的文件描述符为fd1和fd2，则下列叙述中正确的是

I.f1和f2的读写指针位置保持相同

II.f1和f2共享同一个内存索引结点

III.fd1和fd2分别指向各自的用户打开文件表中的一项

A.仅III

B.仅II、III

C.仅I、II

D.I、II和III



文件系统结构

练一练

[2017]若文件f1的硬链接为f2，两个进程分别打开f1和f2，获得对应的文件描述符为fd1和fd2，则下列叙述中正确的是

I.f1和f2的读写指针位置保持相同

II.f1和f2共享同一个内存索引结点

III.fd1和fd2分别指向各自的用户打开文件表中的一项

A.仅III

B.仅II、III



C.仅I、II

D.I、II和III



文件系统结构



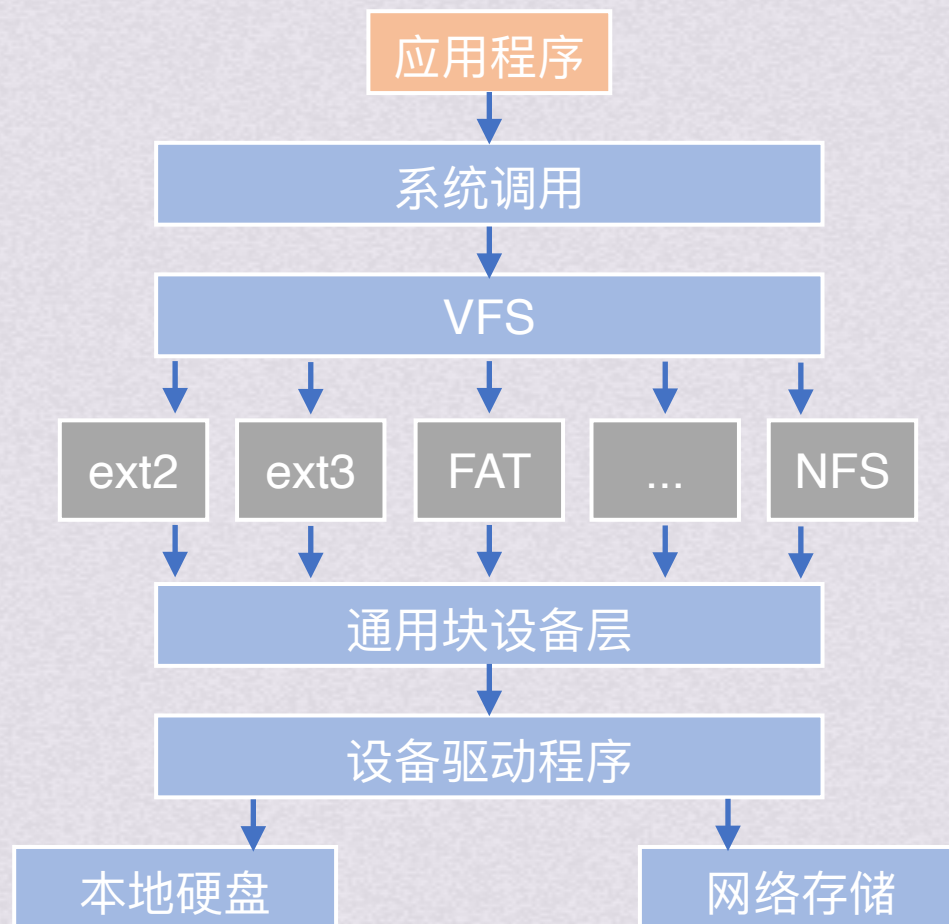
虚拟文件系统



虚拟文件系统

文件系统有很多，ext2,ext3,FAT,NFS等等，但是作为用户在访问文件的时候不需要挠头去了解这些文件系统的具体实现，这就是虚拟文件系统的功劳

虚拟文件系统(VFS)把文件系统的相关操作和不同文件系统的具体细节分离开，为所有的设备提供了统一的接口
用户程序访问文件时，通过VFS提供的系统调用来操作不同文件系统的文件





虚拟文件系统

文件系统有很多，ext2,ext3,FAT,NFS等等，但是作为用户在访问文件的时候不需要挠头去了解这些文件系统的具体实现，这就是虚拟文件系统的功劳

虚拟文件系统(VFS)把文件系统的相关操作和不同文件系统的具体细节分离开，为所有的设备提供了统一的接口

用户程序访问文件时，通过VFS提供的系统调用来操作不同文件系统的文件

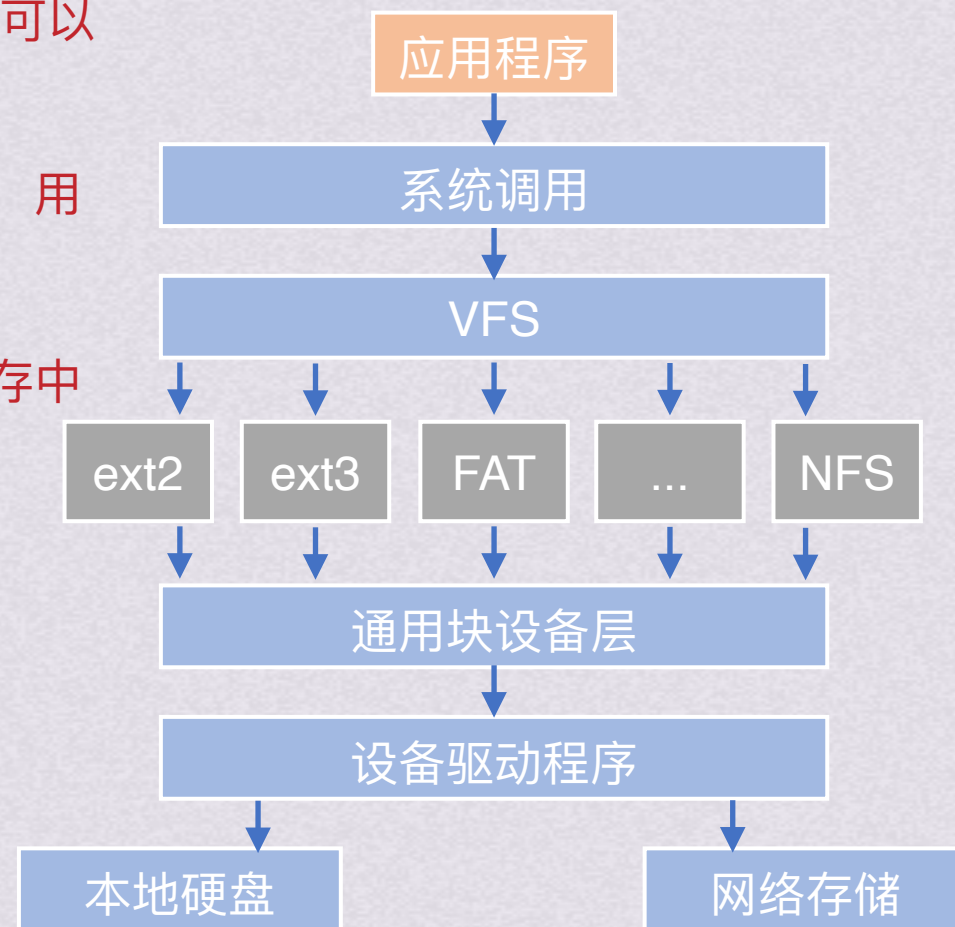
虚拟文件系统抽象出了四种对象类型，新的文件系统只要支持这些接口就可以直接使用

1.超级块对象：标识一个已安装的特定文件系统。对应之前提到的超级块，用来存储已安装文件系统的信息

2.索引节点对象：索引节点和文件是一一对应的关系，文件被访问时才在内存中创建索引节点对象

3.目录项对象：包含关联索引节点的指针和指向父目录、子目录的指针

4.文件对象：表示一个与进程相关的，已打开的文件。





虚拟文件系统

应用程序



系统调用

`open(pathname,flags,mode)`

`pathname`:要打开或创建的文件的路径

`flags`:控制文件打开模式的标志, 如R(只读), W(只写)

`mode`:如果创建新文件, 需指定文件的访问权限

执行过程: 根据指定的路径和标志, 打开或创建一个文件, 返回一个文件描述符

应用程序会使用`open()`、`close()`、`read()`、`write()`的系统调用来完成对文件的访问

`close(fd)`

`fd`:文件描述符, 指向由`open()`打开的文件

执行过程: 关闭文件描述符指向的文件, 释放系统资源

`read(fd,buf,count)`

`fd`: 文件描述符, 指向由`open()`打开的文件

`buf`: 数据读取后存储的缓冲区指针

`count`: 要读取的字节数

执行过程: 从文件描述符指向的文件中读取指定数量的字节到缓冲区, 返回读取的字节数。

`write(fd,buf,count)`

`fd`: 文件描述符, 指向由`open()`打开的文件。

`buf`: 包含要写入文件数据的缓冲区指针。

`count`: 要写入的字节数。

执行过程: 将缓冲区中的数据写入到文件描述符指向的文件, 返回写入的字节数。



虚拟文件系统

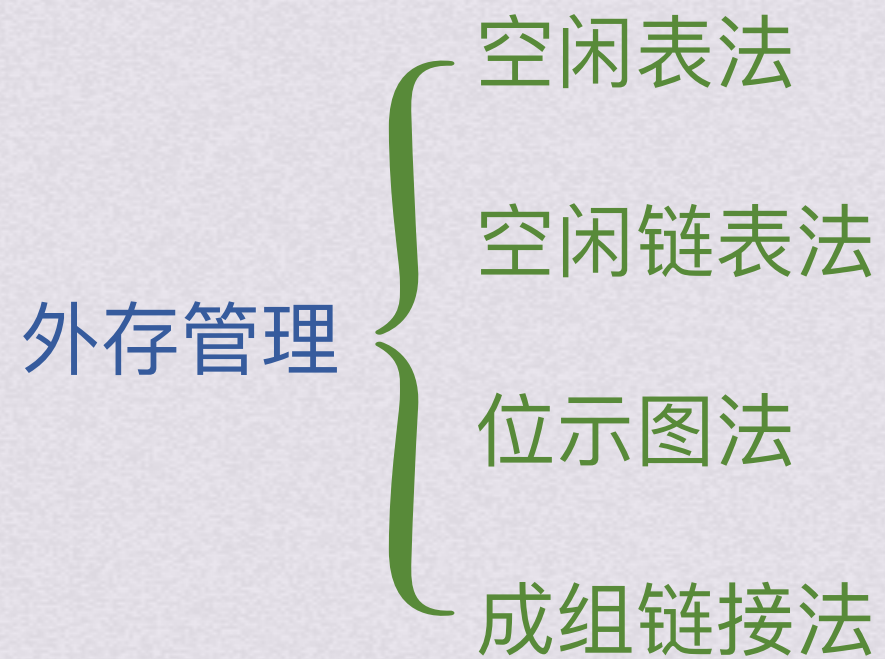


外存管理



外存管理

前面学了连续分配、索引分配、链式分配，这些分配方式都是需要为文件分配盘块的，那么就必须知道磁盘上那些盘块是可以分配的，并且提供盘块分配和回收手段



外存管理

空闲表法

属于连续分配方式，和内存的动态分配方式差不多，为每个文件分配一块连续的存储空间，再为所有外存的空闲区域建立一张空闲表

序号	第一空闲盘块号	空闲盘块数
1	2	4
2	9	3
3	15	5
4	...	...

盘块的分配和回收也和内存的动态分配相似，首次适应算法、最佳适应算法等

空闲表法适用于连续分配，所以容易得到连续的存储空间



空闲表法



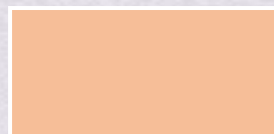
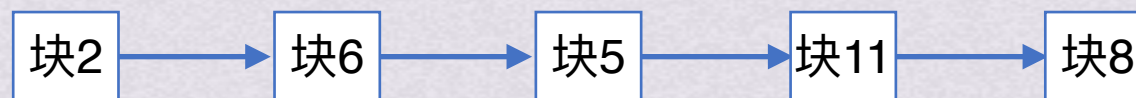
空闲链表法



空闲链表法

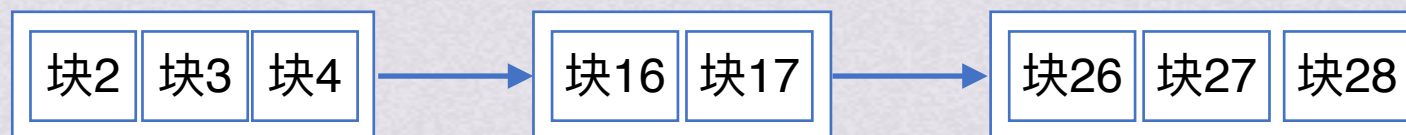
将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链

空闲盘块链



文件申请两块大小的空间

空闲盘区链

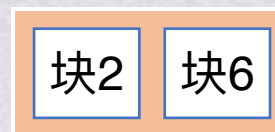
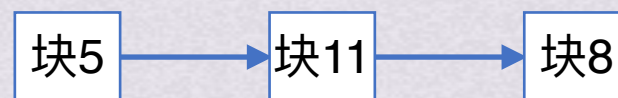




空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链

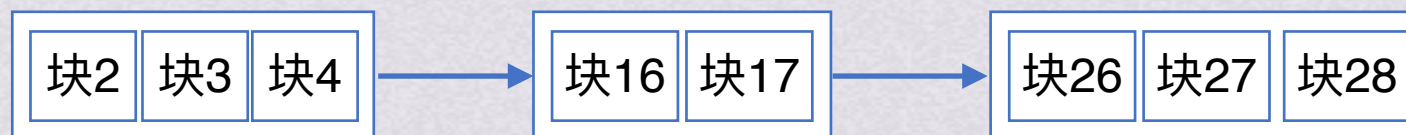
空闲盘块链



文件申请两块大小的空间

空闲盘区链

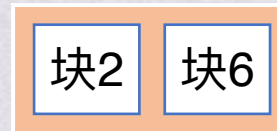
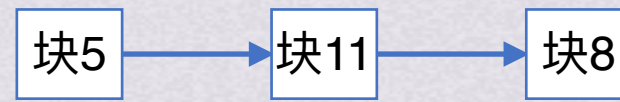
(假设首次适应算法)



空闲链表法

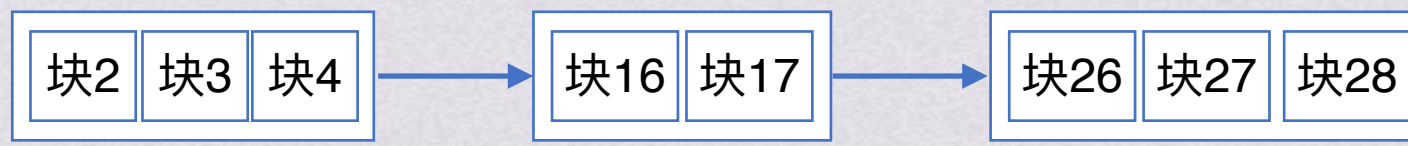
将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链

空闲盘块链



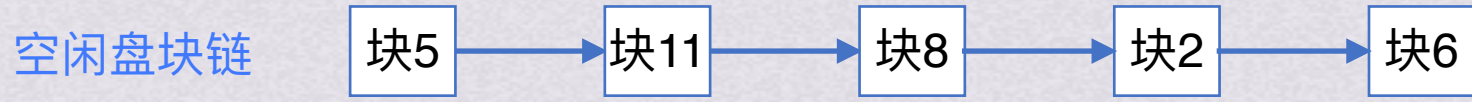
文件申请两块大小的空间
文件释放空间

空闲盘区链
(假设首次适应算法)

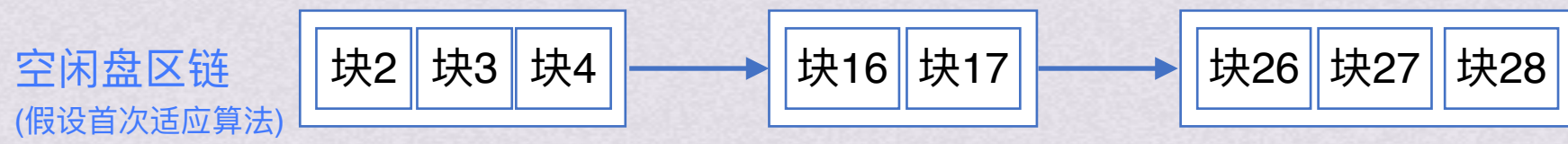


空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链



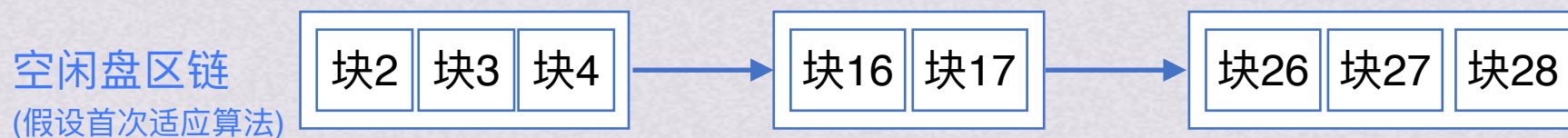
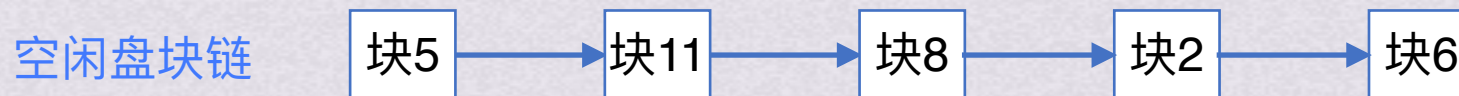
文件申请两块大小的空间
文件释放空间





空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链

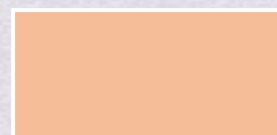
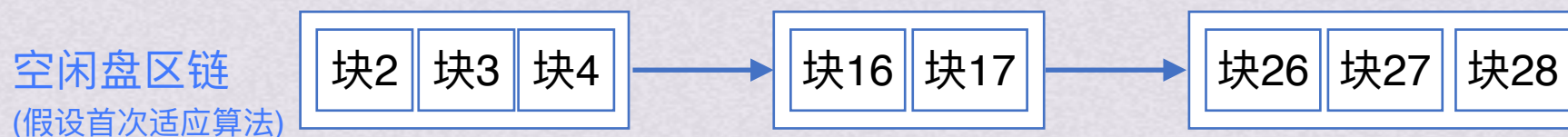
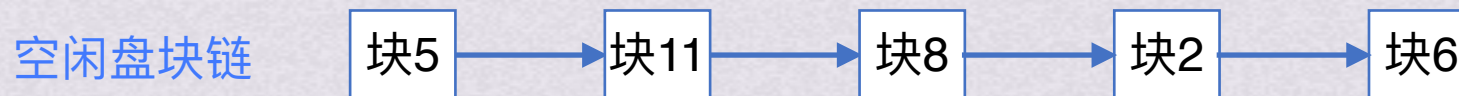


文件申请两块大小的空间



空闲链表法

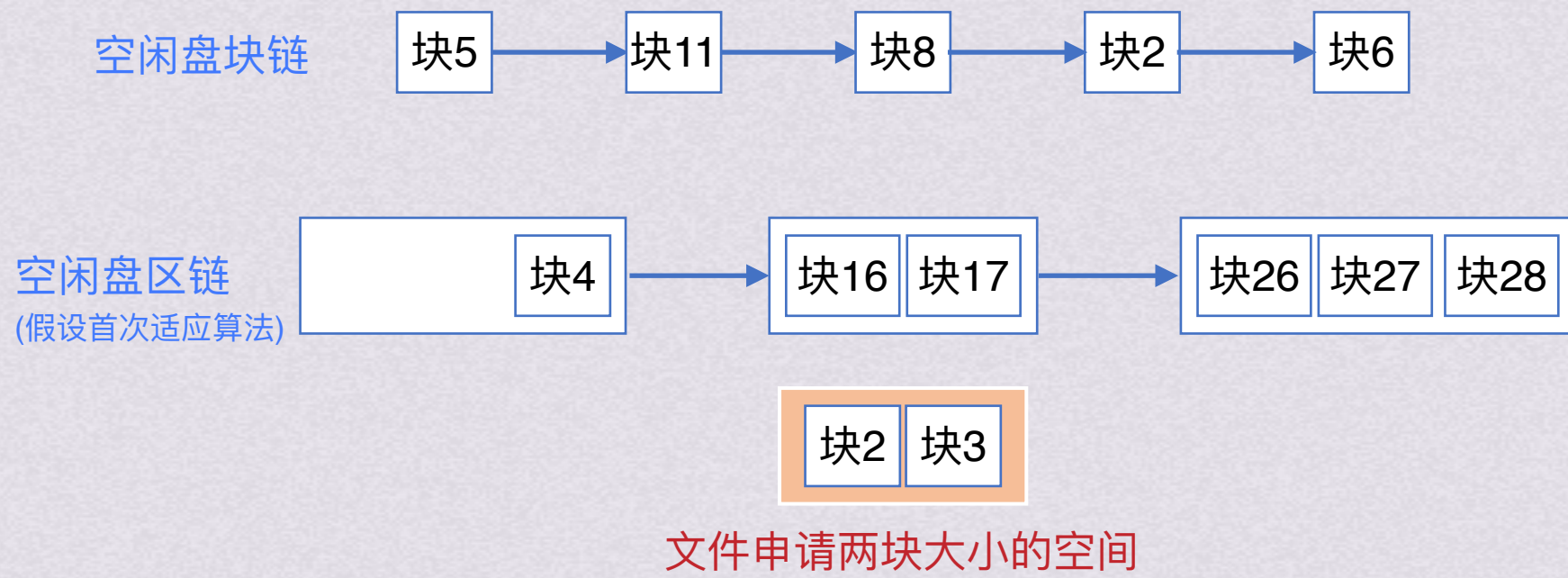
将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链



文件申请两块大小的空间

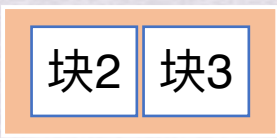
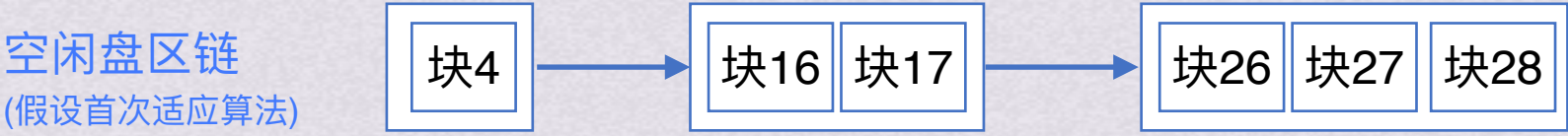
空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链



空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链

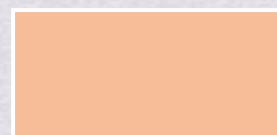
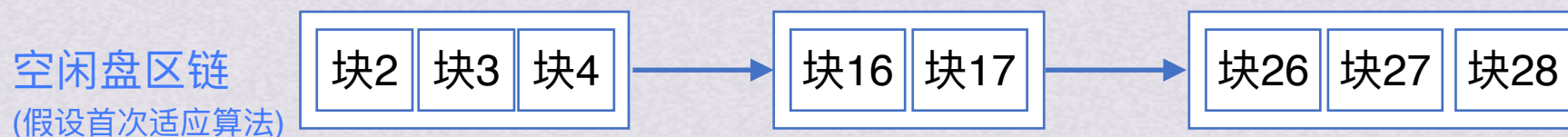
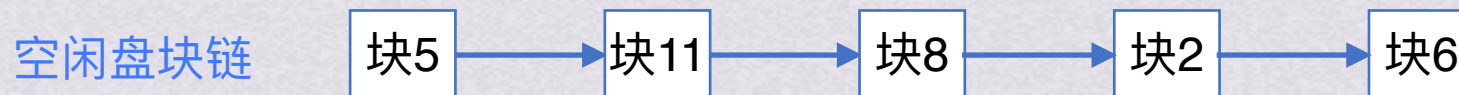


文件申请两块大小的空间
文件释放空间



空闲链表法

将所有空闲盘区拉成一条空闲链，根据构成链条元素，分成空闲盘块链和空闲盘区链



文件申请两块大小的空间

文件释放空间

空闲链表法适用于离散分配，比较难得到连续空间



空闲链表法

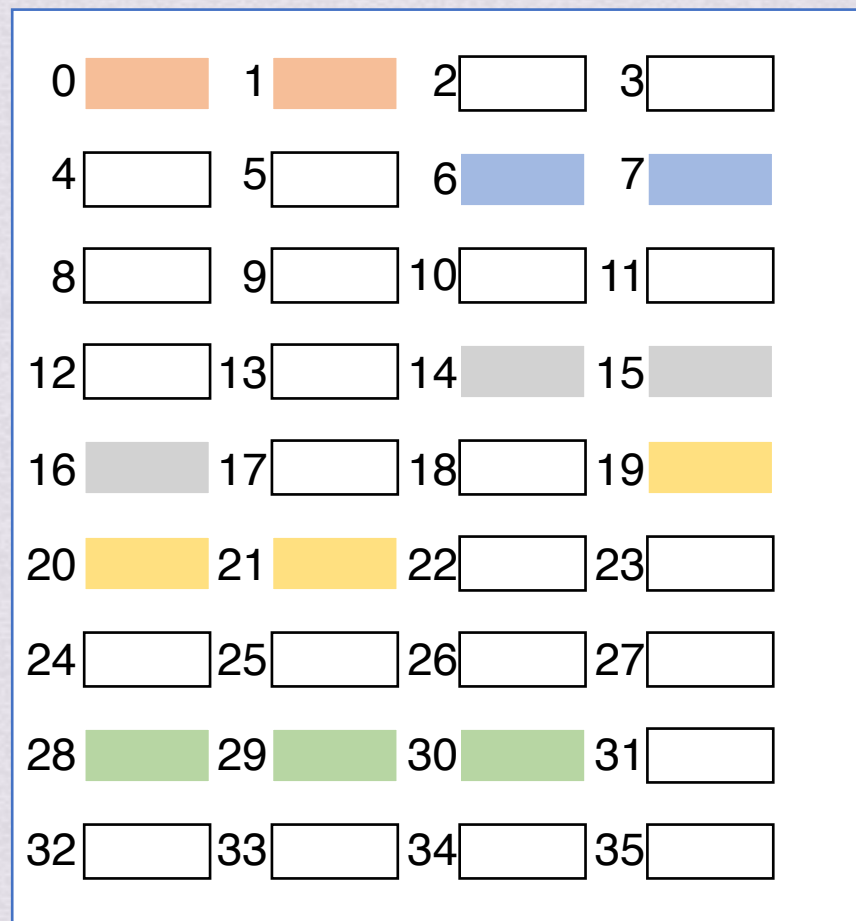


位示图法



位示图法

利用二进制的0表示盘块空闲，1表示盘块已分配(反着也行)，这样所有盘块对应的比特构成一个集合，叫做位示图



盘块情况

1	1	0	0
0	0	1	1
0	0	0	0
0	0	1	1
1	0	0	1
1	1	0	0
0	0	0	0
1	1	1	0
0	0	0	0

对应位示图

位示图适用于连续分配和离散分配，容易找到连续的空闲块



位示图法

练一练

[2014]现有一个容量为10GB的磁盘分区，磁盘空间以簇为单位进行分配，簇的大小为4KB，若采用位图法管理该分区的空闲空间，即用一位来标识一个簇是否被分配，则存放该位图所需要的簇数量为()

A.80

B.320

C.80K

D.320K



位示图法

练一练

[2014]现有一个容量为10GB的磁盘分区，磁盘空间以簇为单位进行分配，簇的大小为4KB，若采用位图法管理该分区的空闲空间，即用一位来标识一个簇是否被分配，则存放该位图所需要的簇数量为()

A.80

B.320

C.80K

D.320K



位示图法

练一练

[2014]现有一个容量为10GB的磁盘分区，磁盘空间以簇为单位进行分配，簇的大小为4KB，若采用位图法管理该分区的空闲空间，即用一位来标识一个簇是否被分配，则存放该位图所需要的簇数量为()

每个簇需要1bit来标识是否被分配，一共有 $\frac{10GB}{4KB} = 2.5M$ 个簇

也就需要2.5M个比特来记录簇是否被分配,也就是 $\frac{2.5M}{8} = 320KB$ 来记录

一个簇能存放4KB，320KB需要80个簇

A.80

B.320

C.80K

D.320K



位示图法

练一练

[2015]文件系统用位图法标识磁盘空间的分配情况，位图存在于磁盘的32~127号块中，每个盘块占1024B，盘块和块内字节均从0开始编号。假设要释放的盘块号为409612，则位图中要修改的位所在的盘块号和块内字节序号分别是()

A.81,1

B.81,2

C.82,1

D.82,2



位示图法

练一练

[2015]文件系统用位图法标识磁盘空间的分配情况，位图存在于磁盘的32~127号块中，每个盘块占1024B，盘块和块内字节均从0开始编号。假设要释放的盘块号为409612，则位图中要修改的位所在的盘块号和块内字节序号分别是()

A.81,1

B.81,2

C.82,1

D.82,2



位示图法

练一练

[2015]文件系统用位图法标识磁盘空间的分配情况，位图存在于磁盘的32~127号块中，每个盘块占1024B，盘块和块内字节均从0开始编号。假设要释放的盘块号为409612，则位图中要修改的位所在的盘块号和块内字节序号分别是()

盘块号409612，从0开始计数，那么409612是第409613个数字

$$409613 = \frac{1024 \times 8 \times 50}{409600} + 13$$

即这么多数字能放满50个盘块，还要多出13个数字，也就是需要占用51个盘块

从32开始的第51个盘块是82号盘块

$$x - 32 + 1 = 51 \rightarrow x = 82$$

还多出来了13个数字， $13 = 8 \times 1 + 5$ ，占用一个字节多一点，所以序号是1(因为序号从0开始)



位示图法



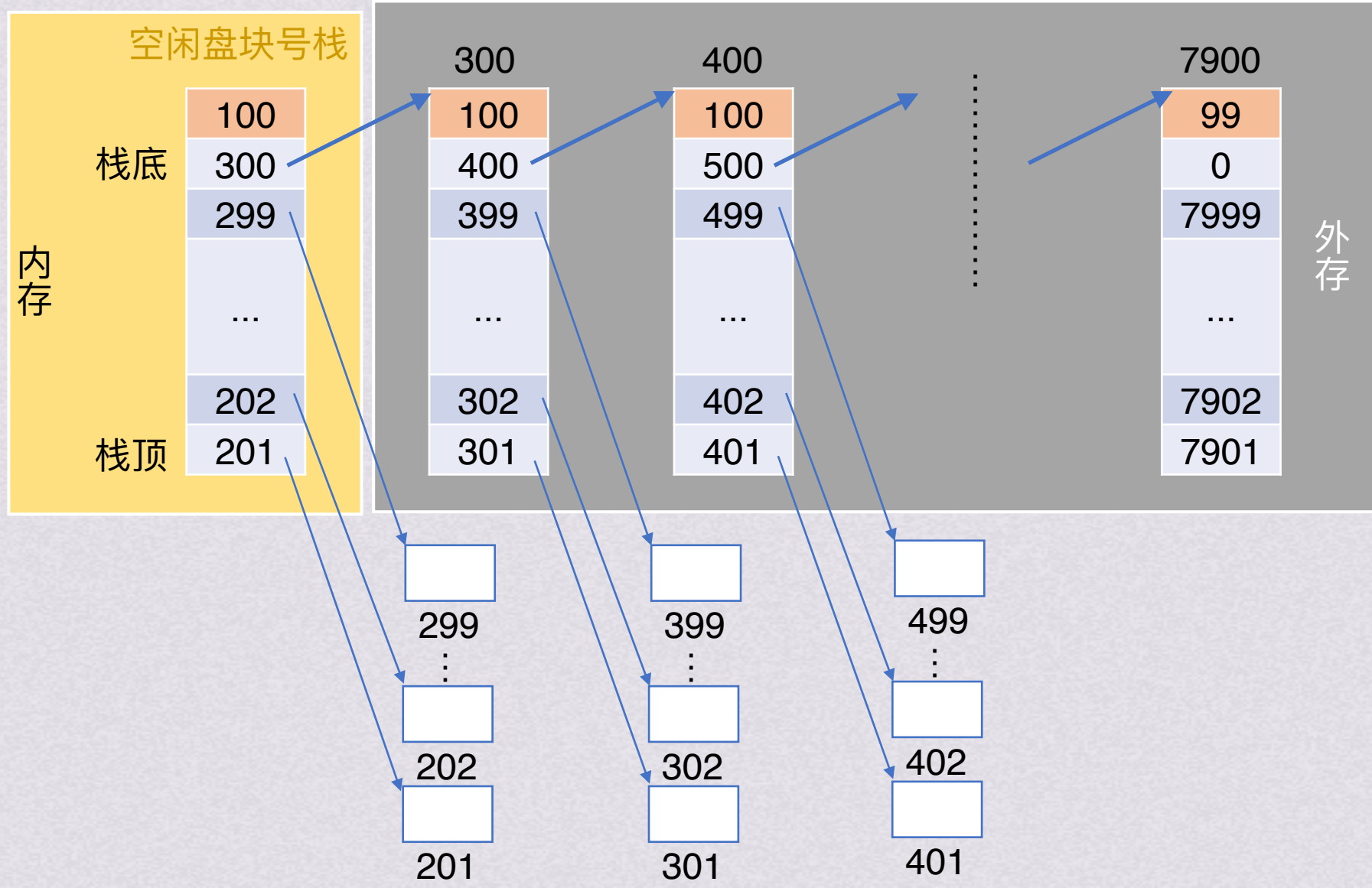
成组链接法



外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

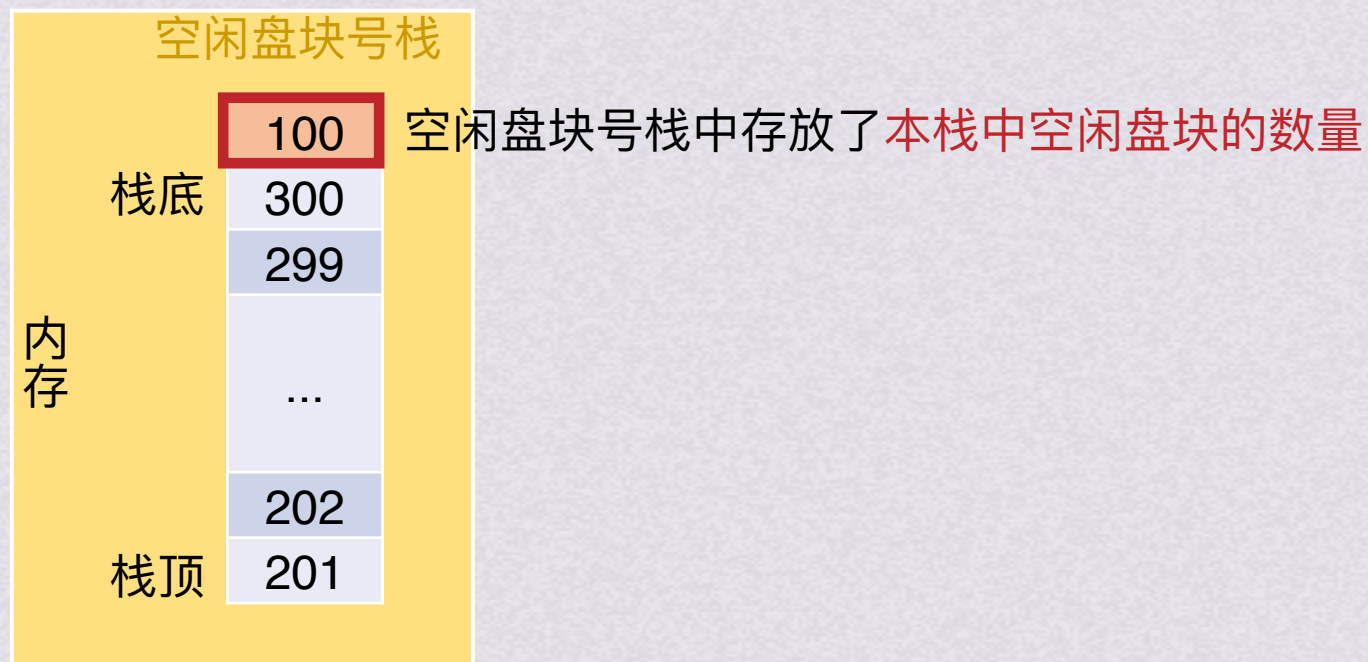




外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

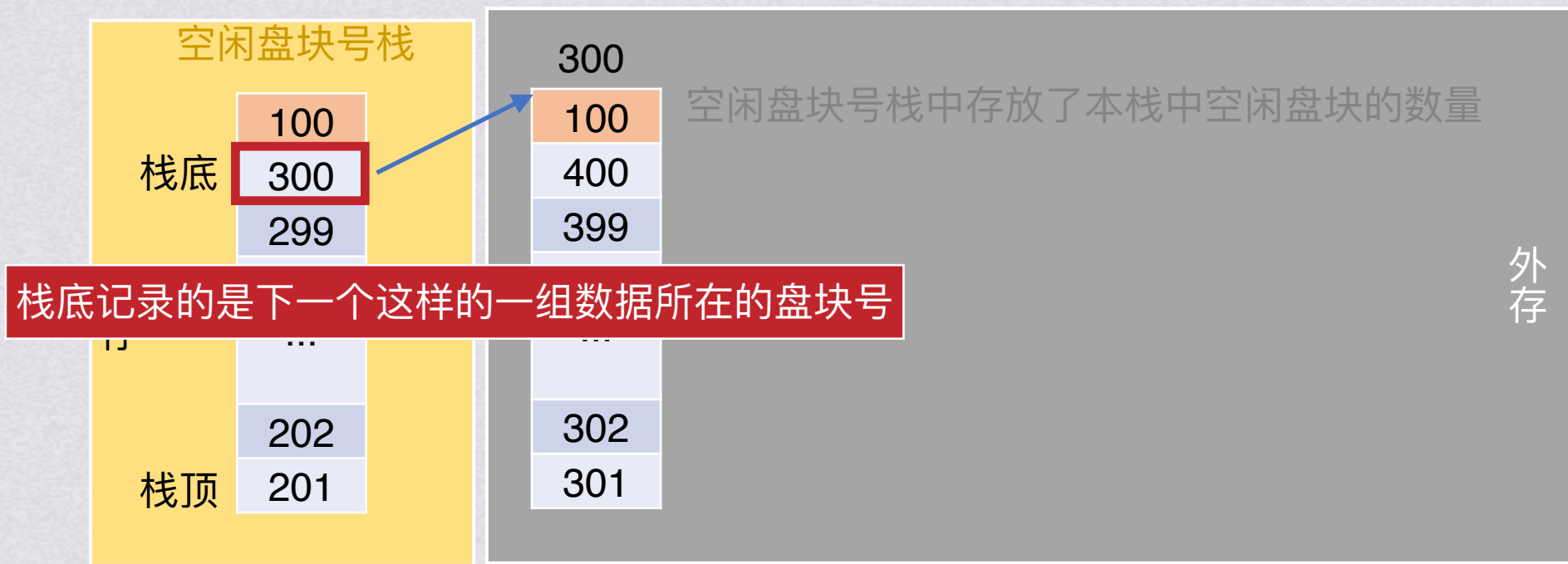




外存管理

成组链接法

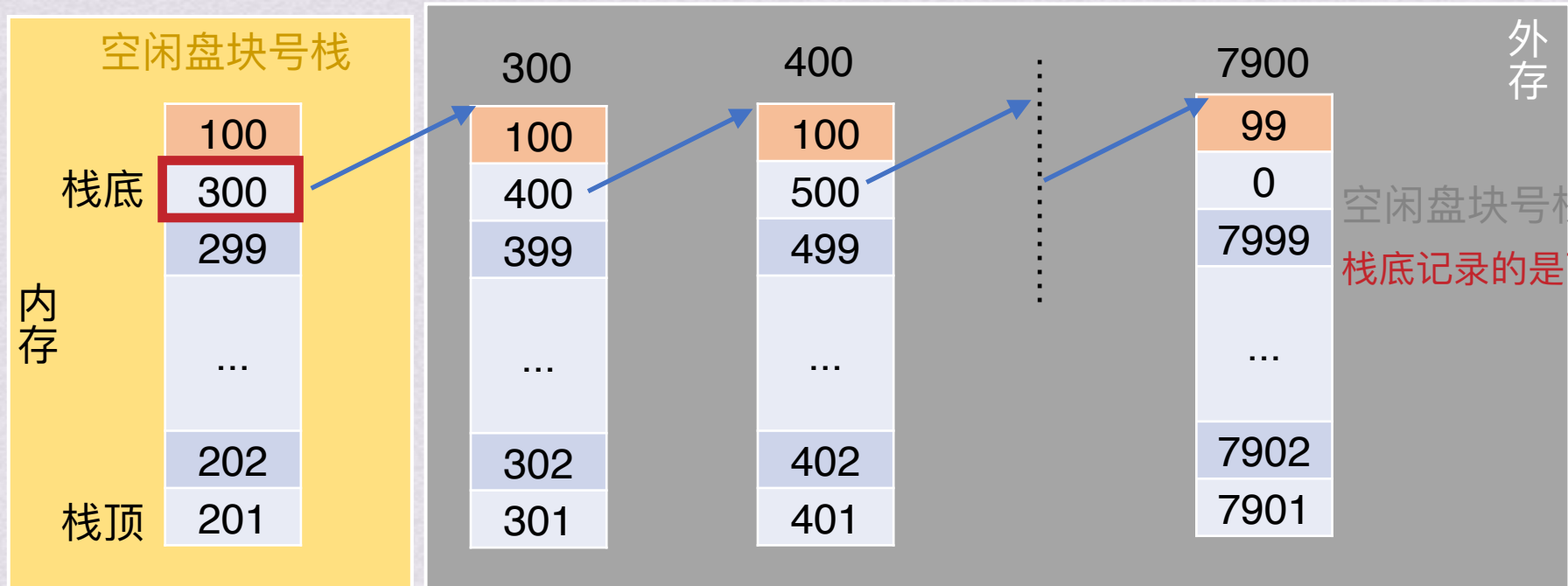
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

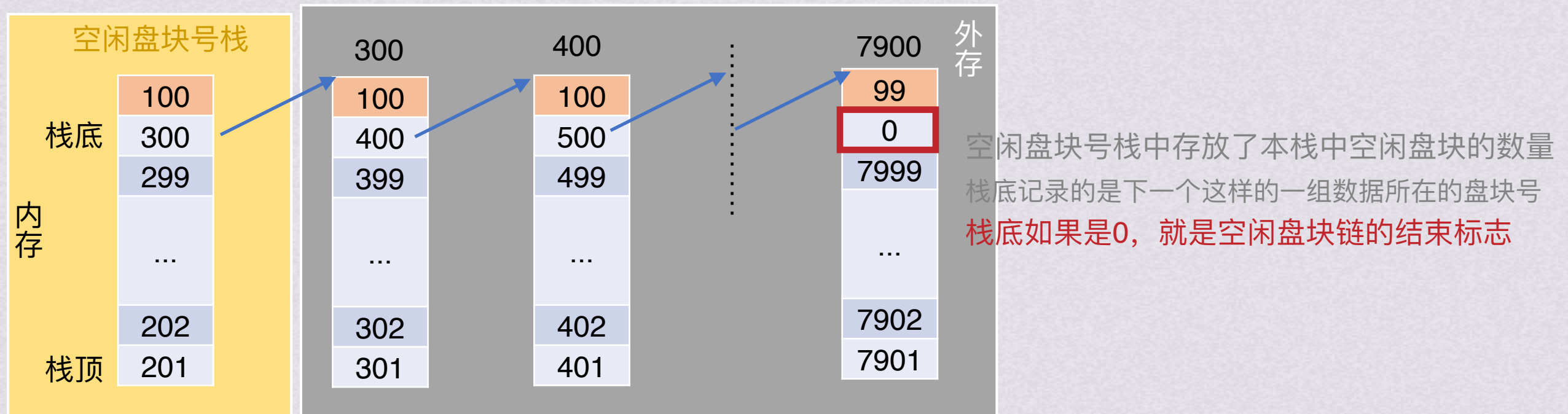


空闲盘块号栈中存放了本栈中空闲盘块的数量
栈底记录的是下一个这样的一组数据所在的盘块号

外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

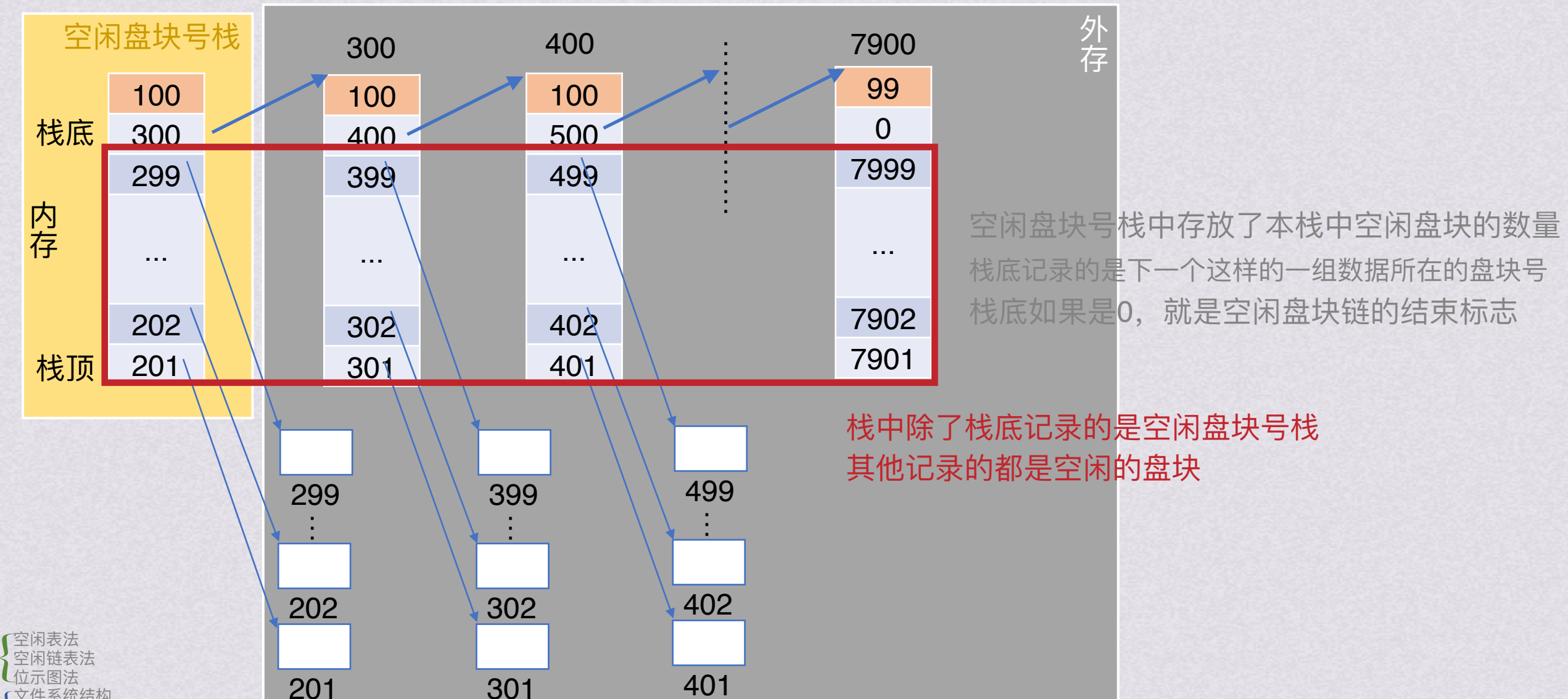




外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

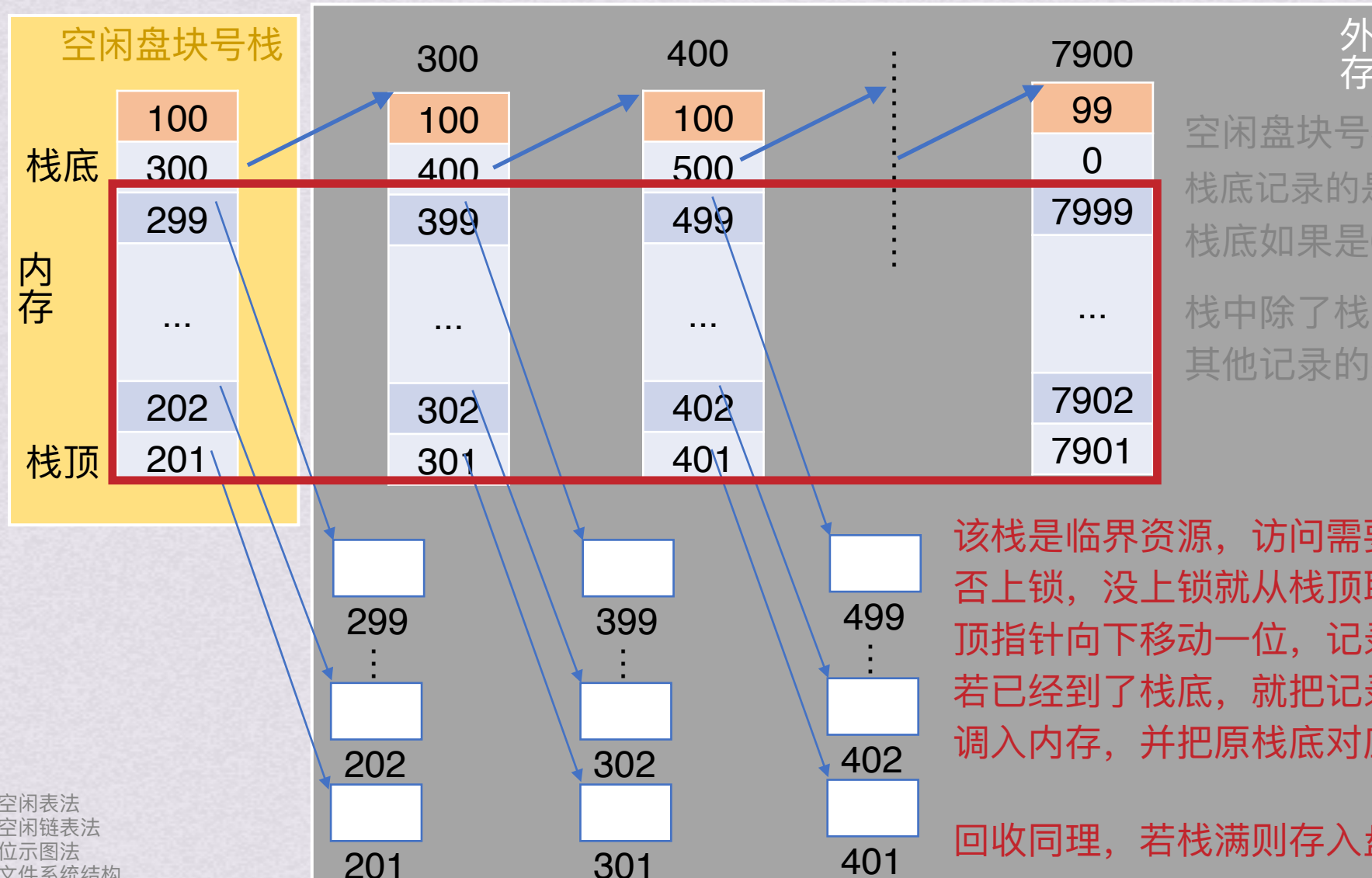




外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



空闲盘块号栈中存放了本栈中空闲盘块的数量
栈底记录的是下一个这样的一组数据所在的盘块号
栈底如果是0，就是空闲盘块链的结束标志

栈中除了栈底记录的是空闲盘块号栈
其他记录的都是空闲的盘块

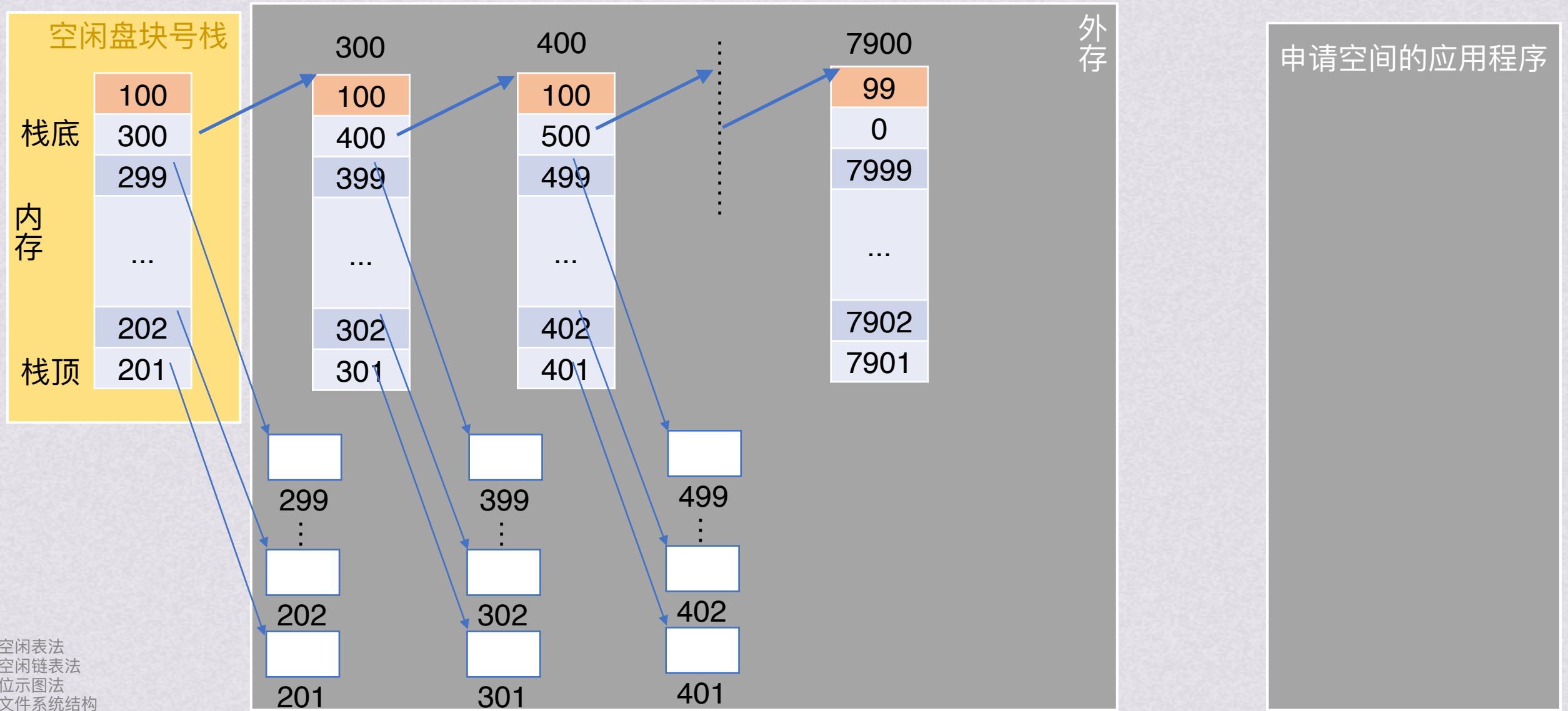
该栈是临界资源，访问需要互斥，分配盘块时，先检查栈是否上锁，没上锁就从栈顶取出空闲盘块号分给用户，然后栈顶指针向下移动一位，记录空闲盘块的数量--
若已经到了栈底，就把记录的下一个空闲盘块号栈所在的块调入内存，并把原栈底对应盘块分配出去

回收同理，若栈满则存入盘块，再找新盘块作为栈底

外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

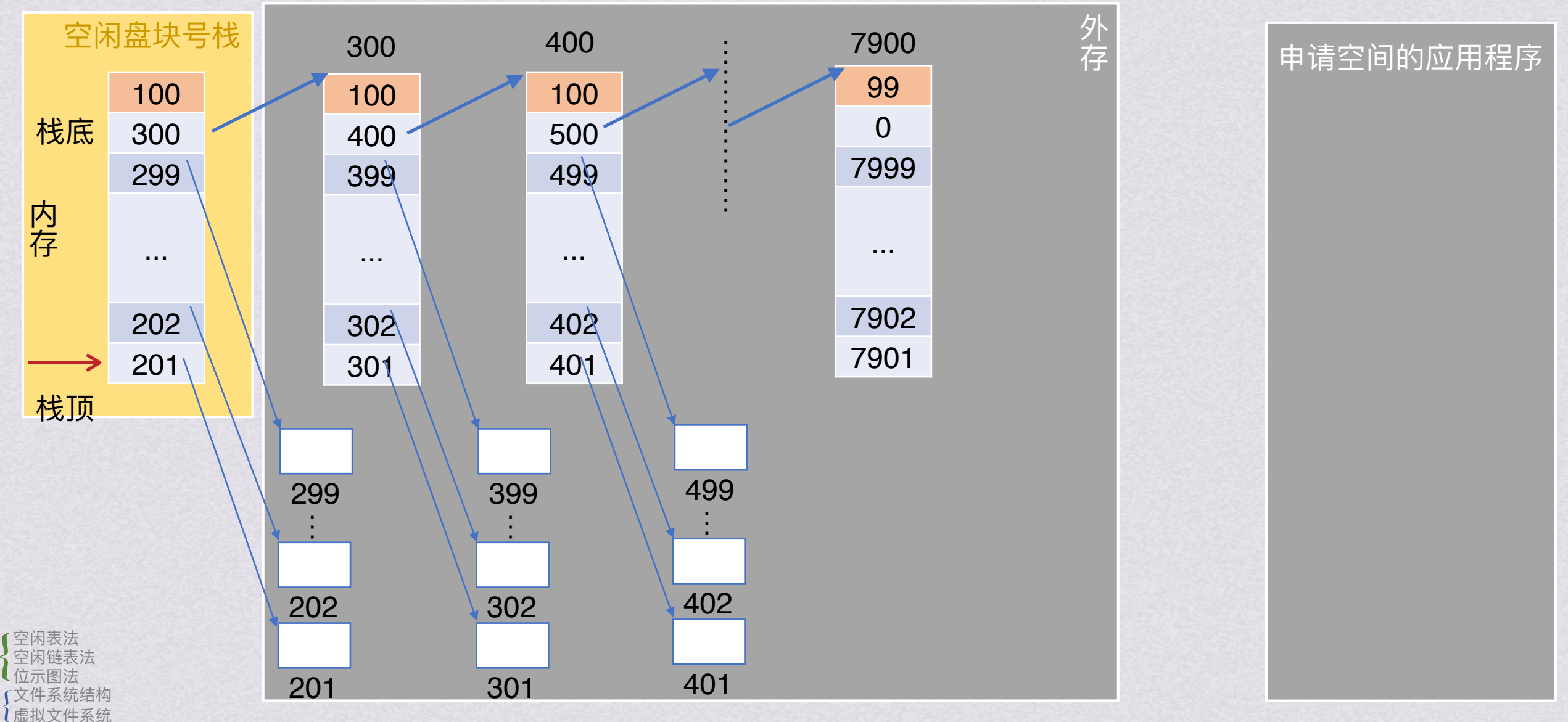




外存管理

成组链接法

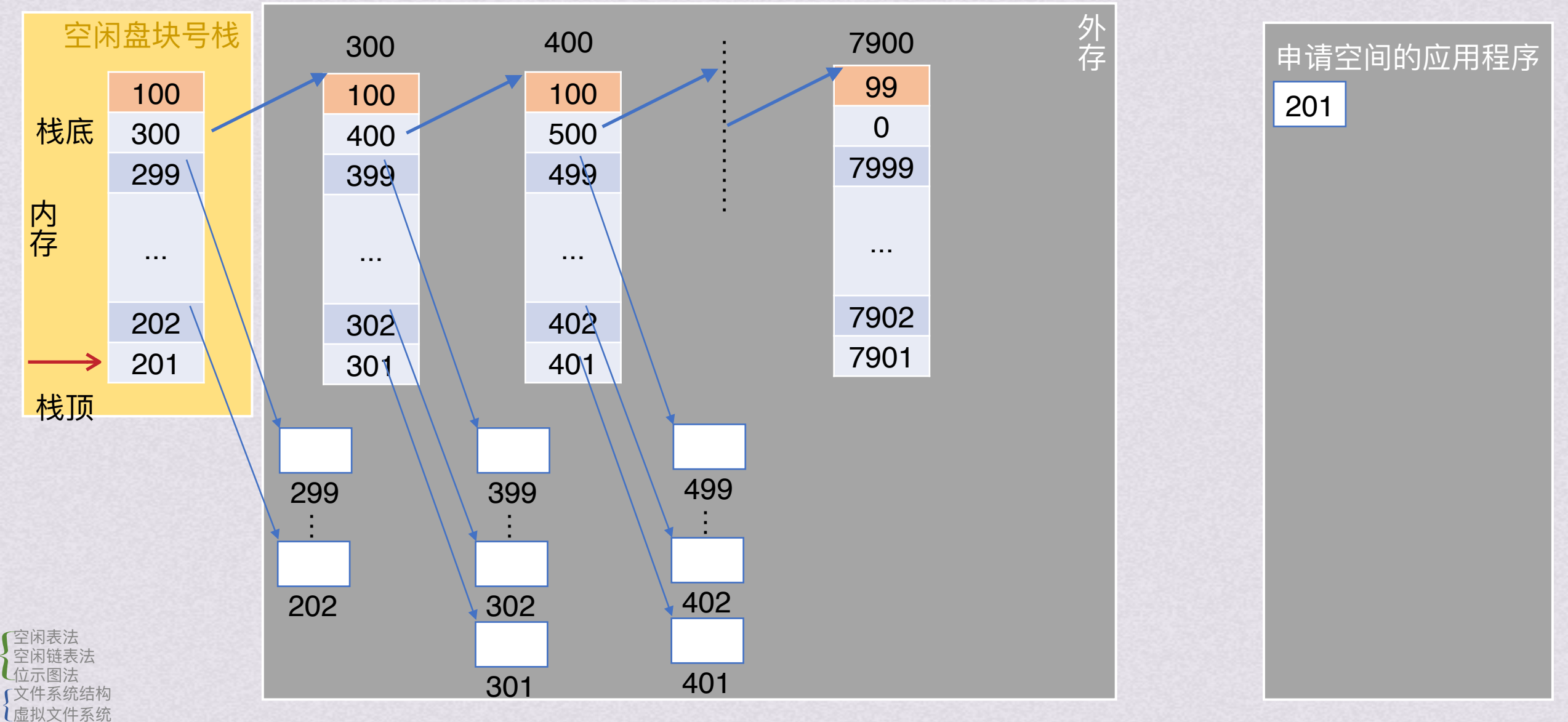
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

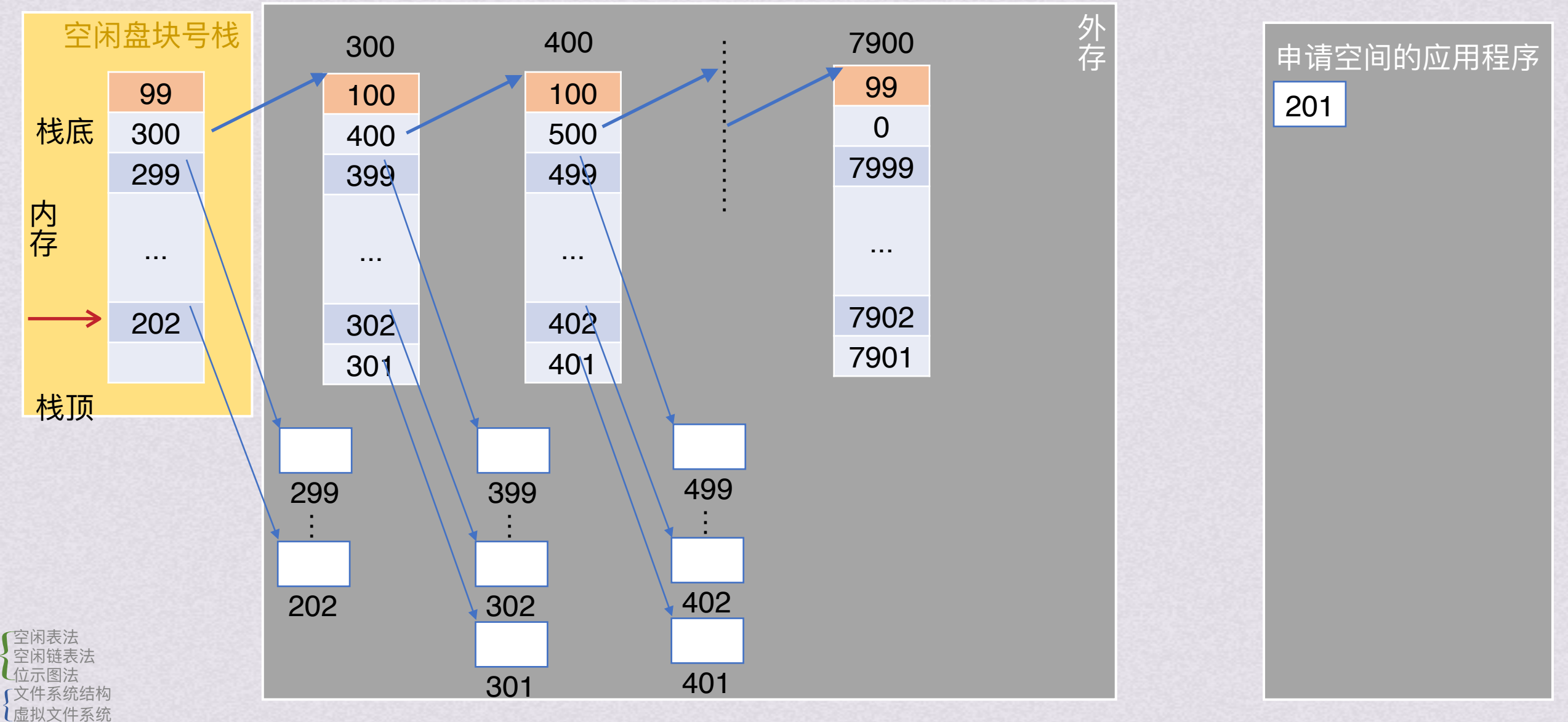
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

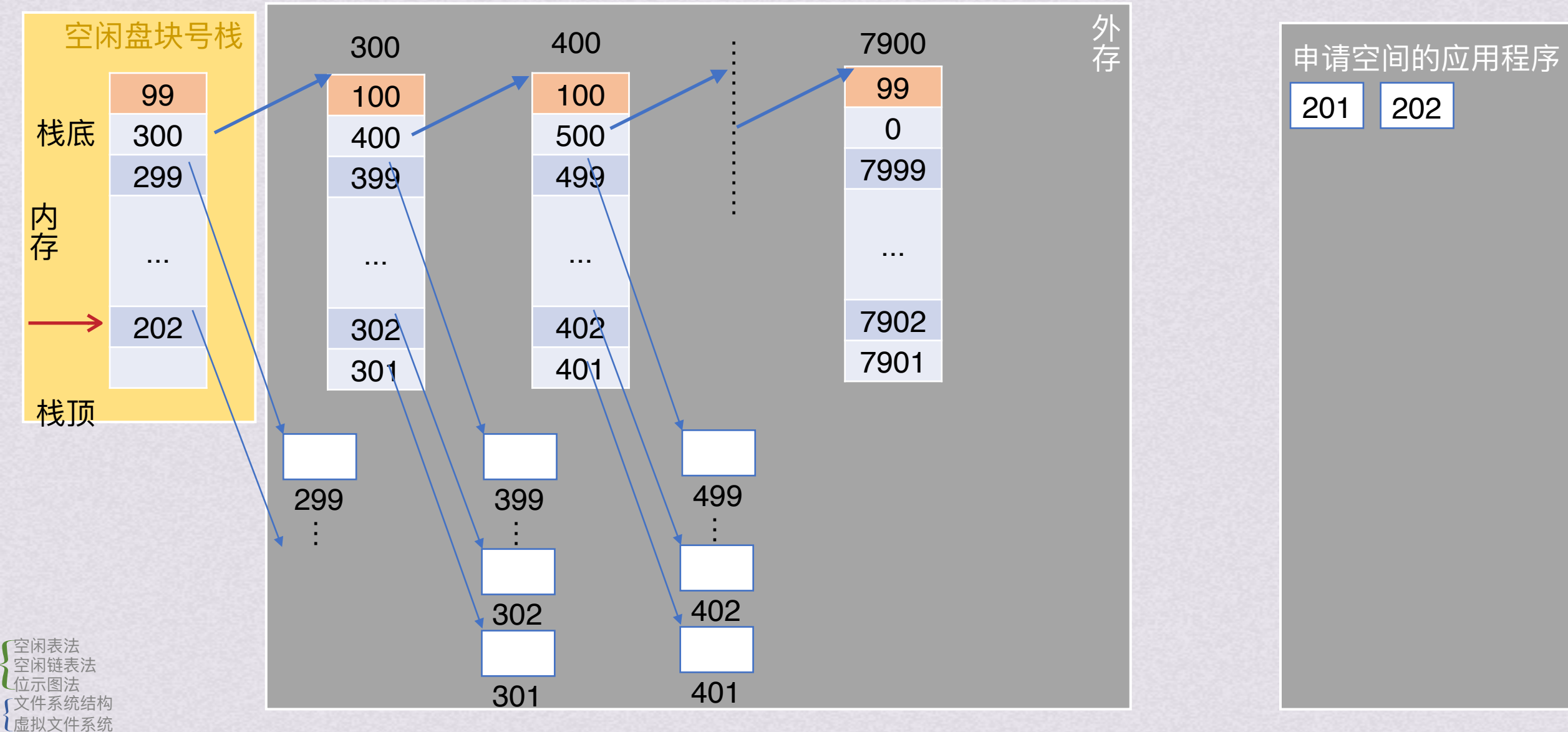




外存管理

成组链接法

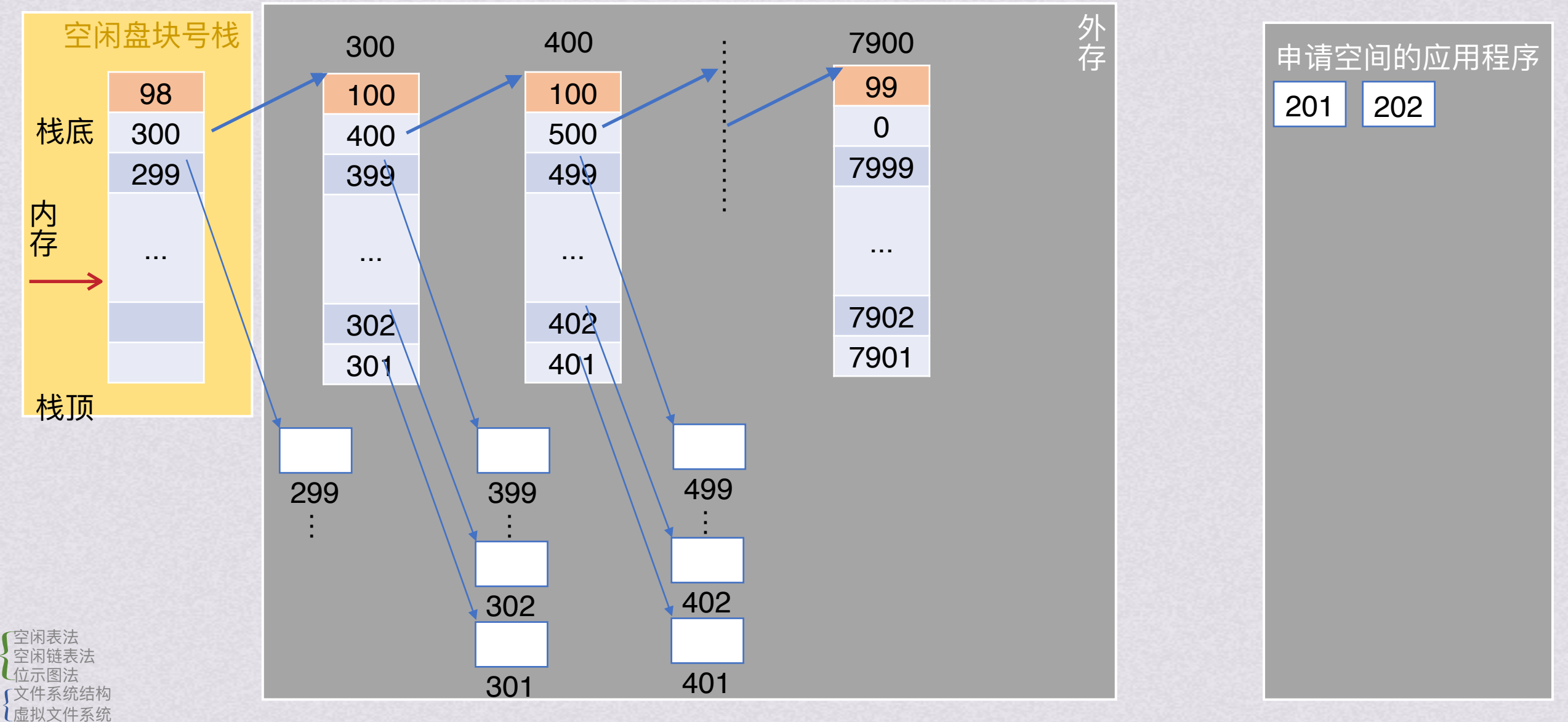
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

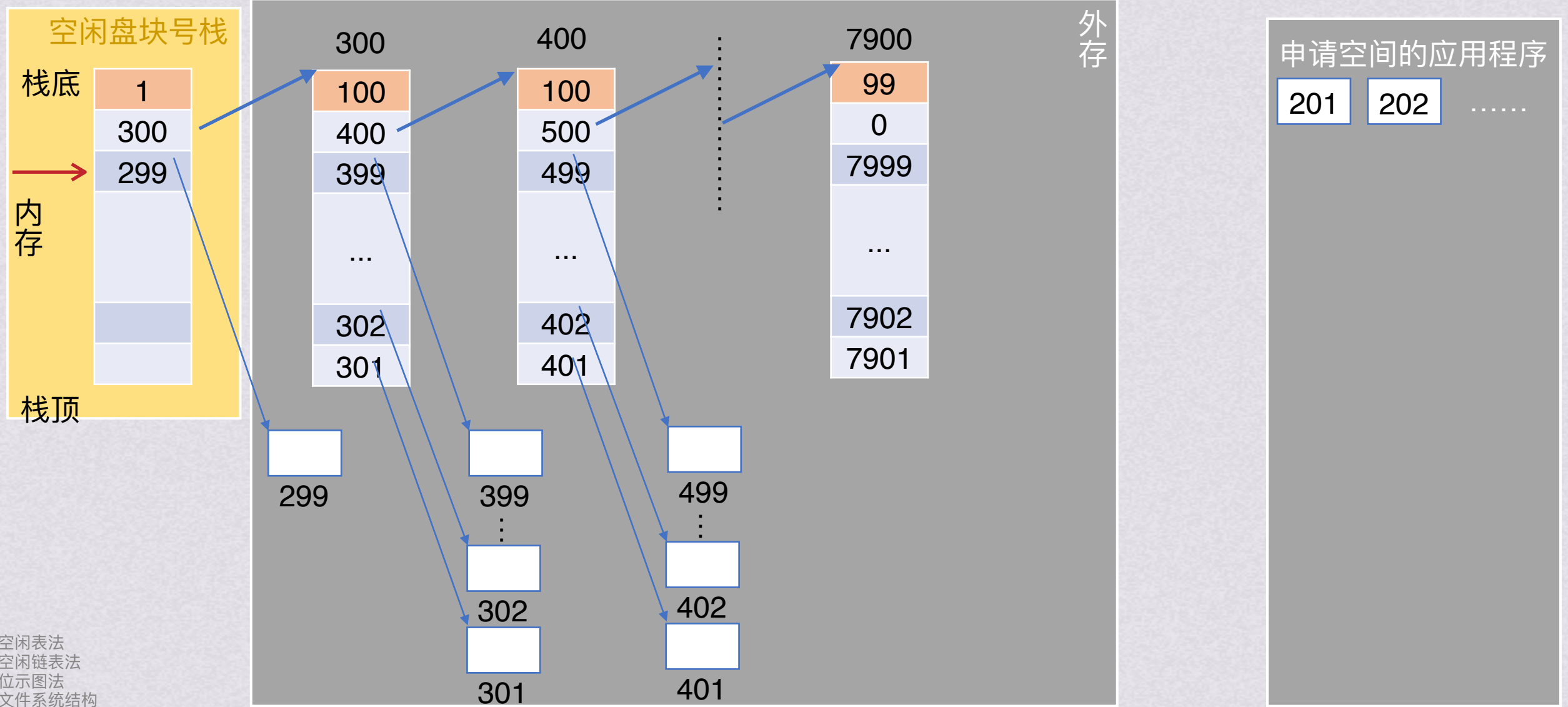
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

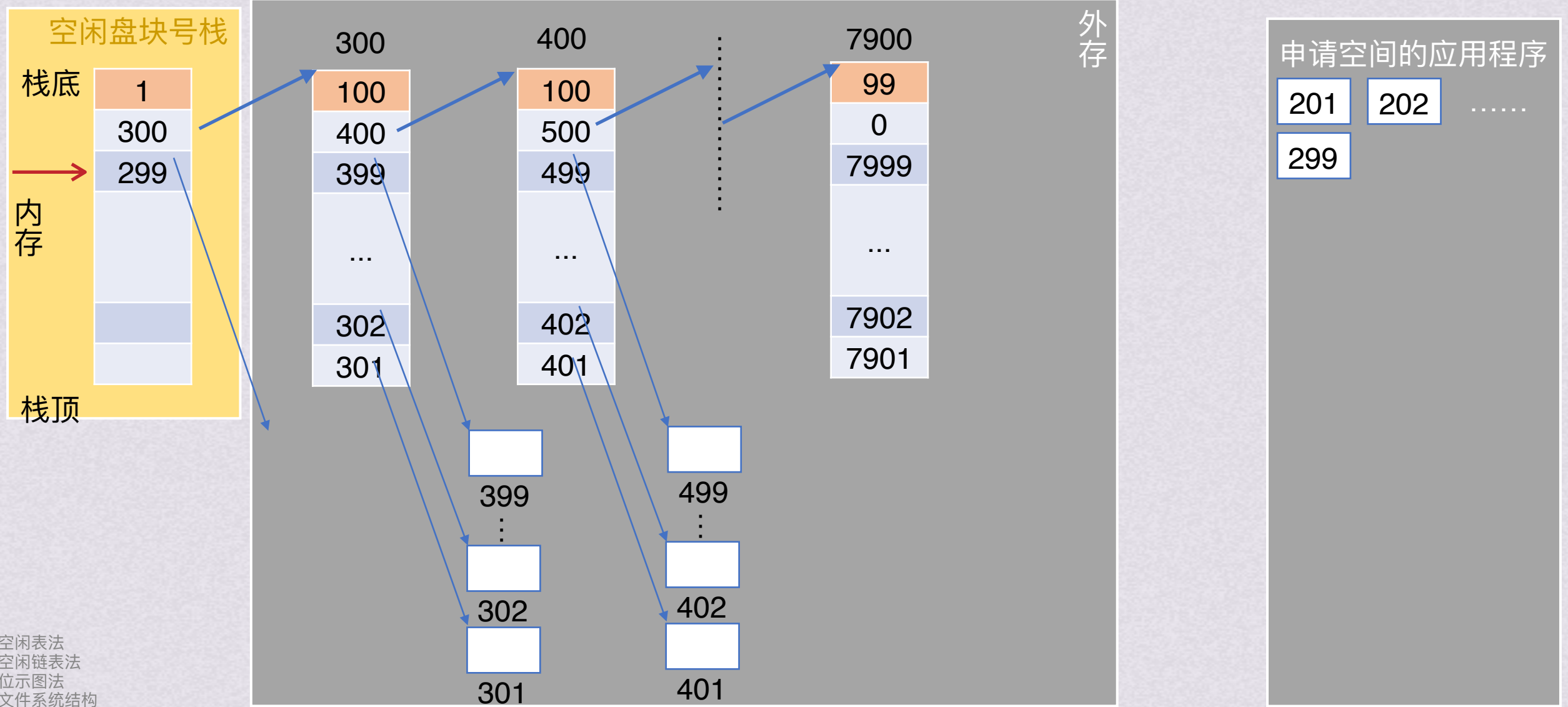
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

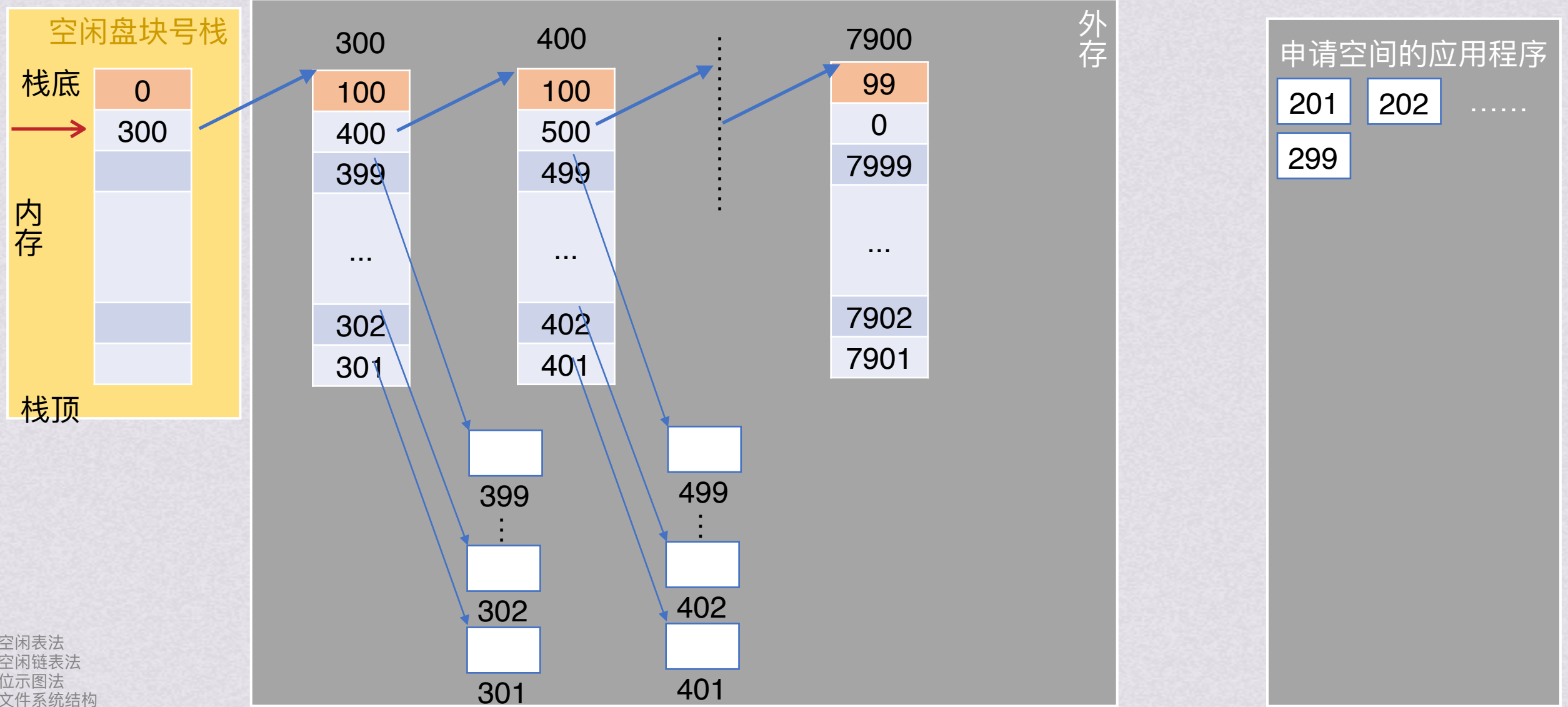
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

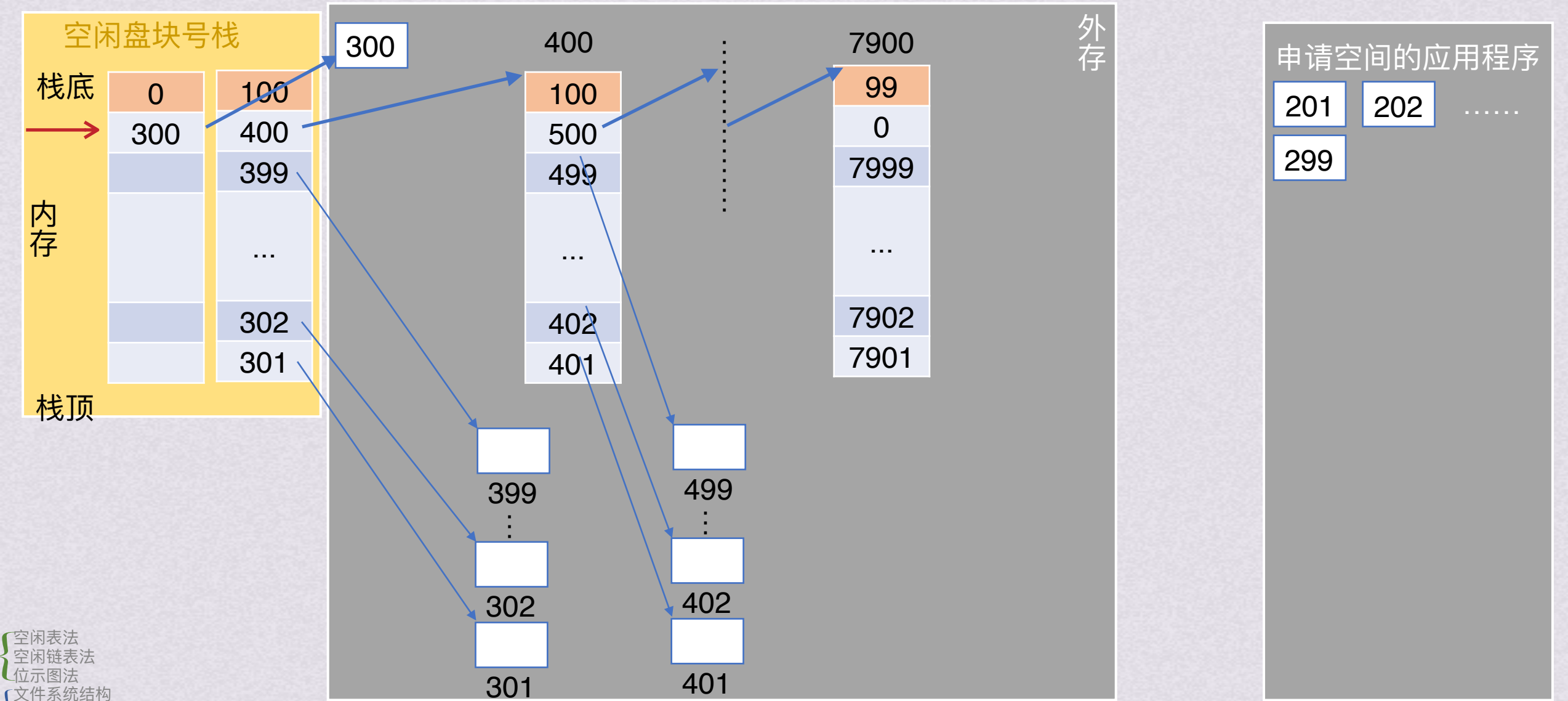
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

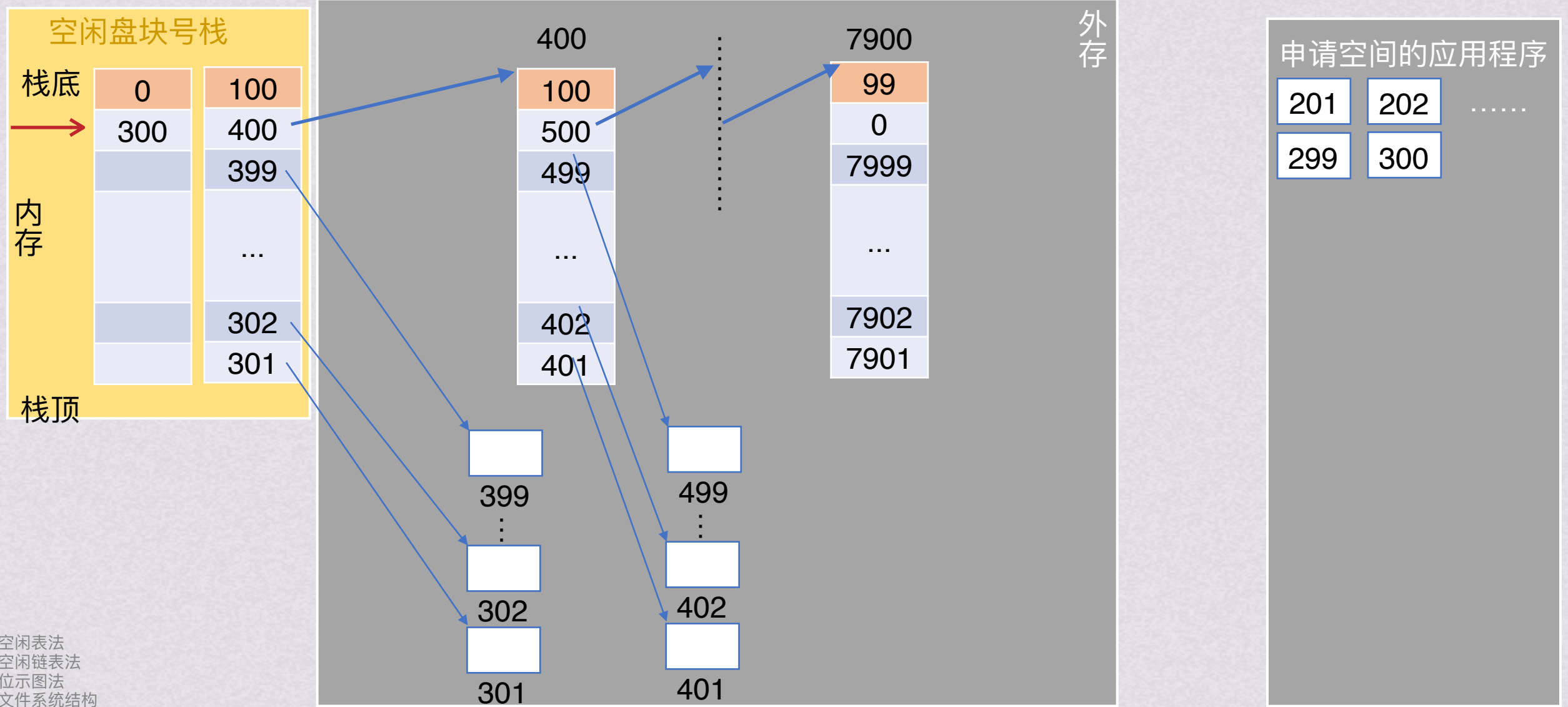
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

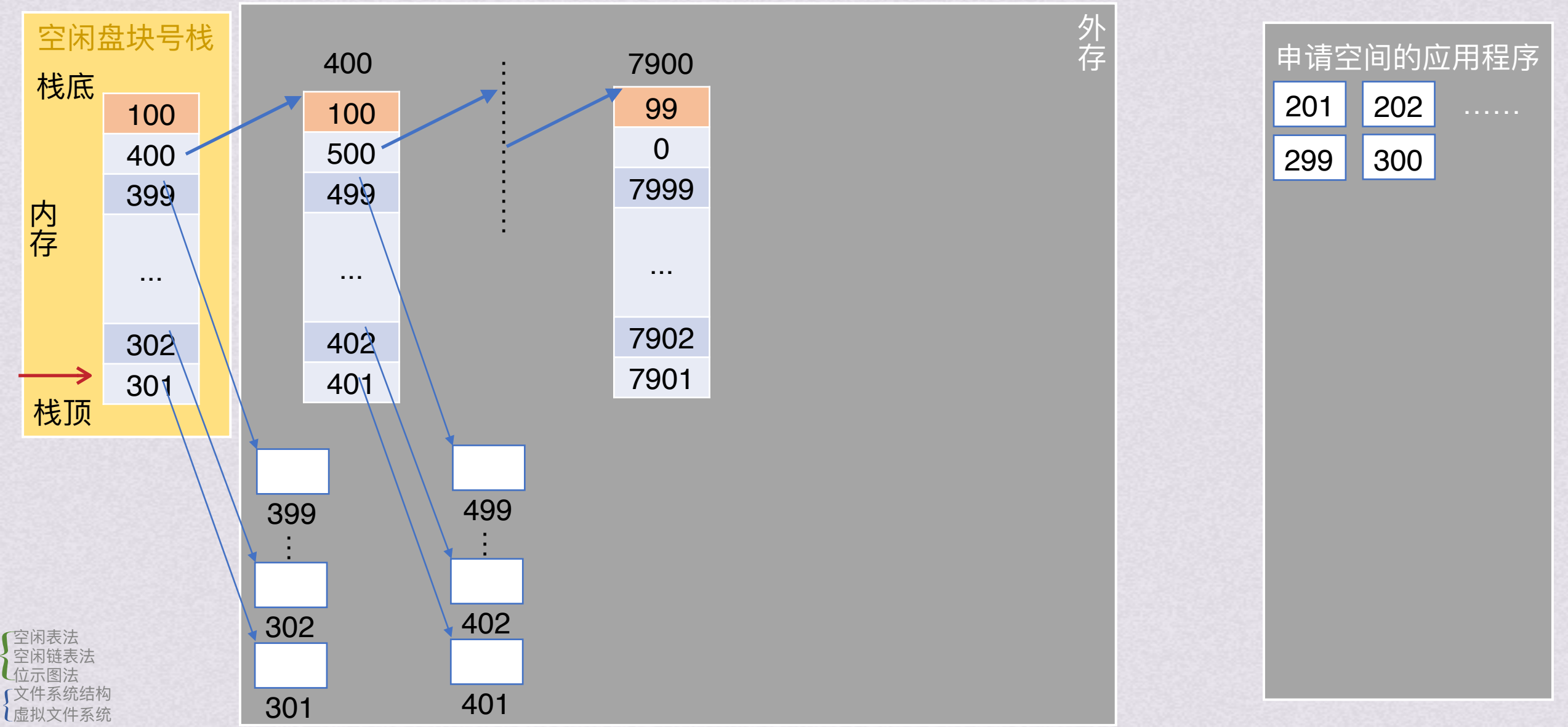
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

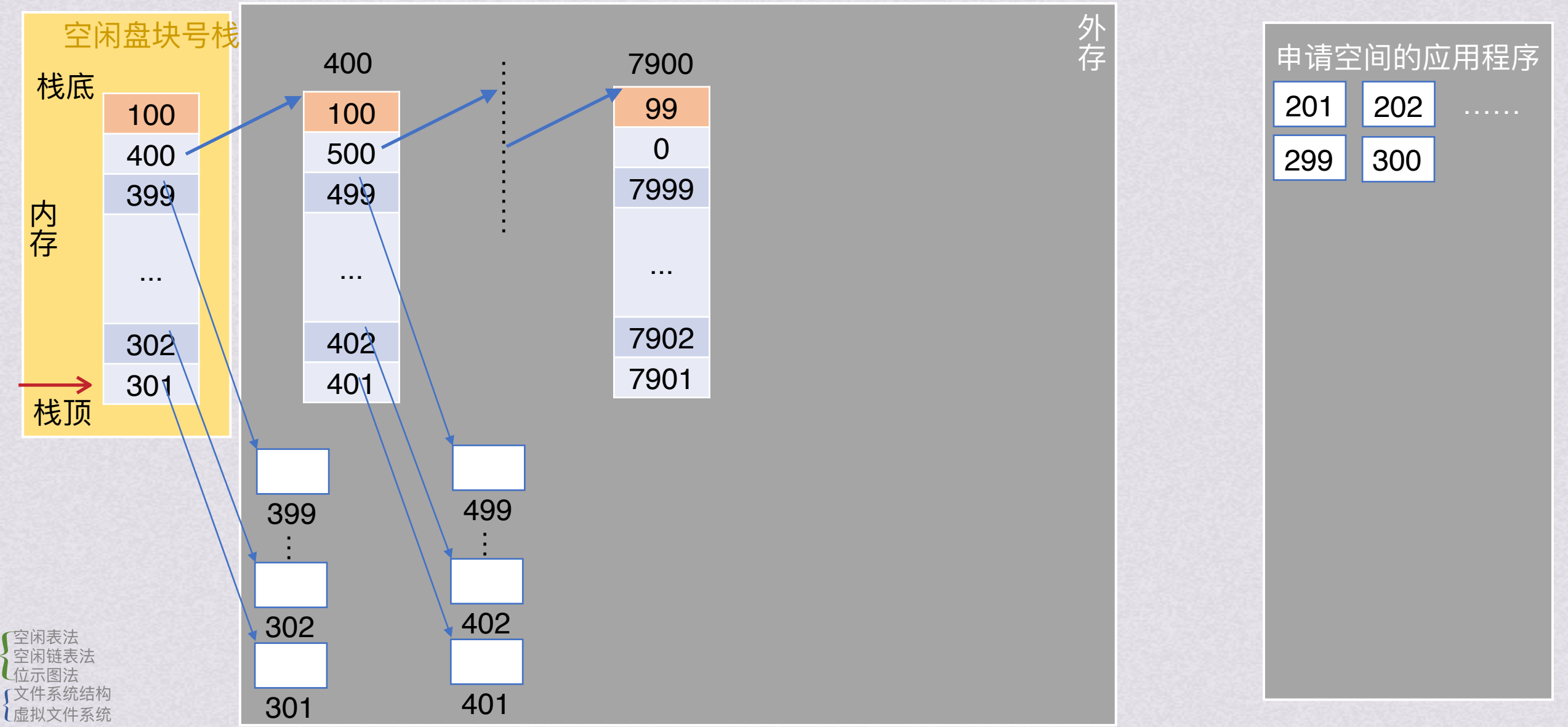
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

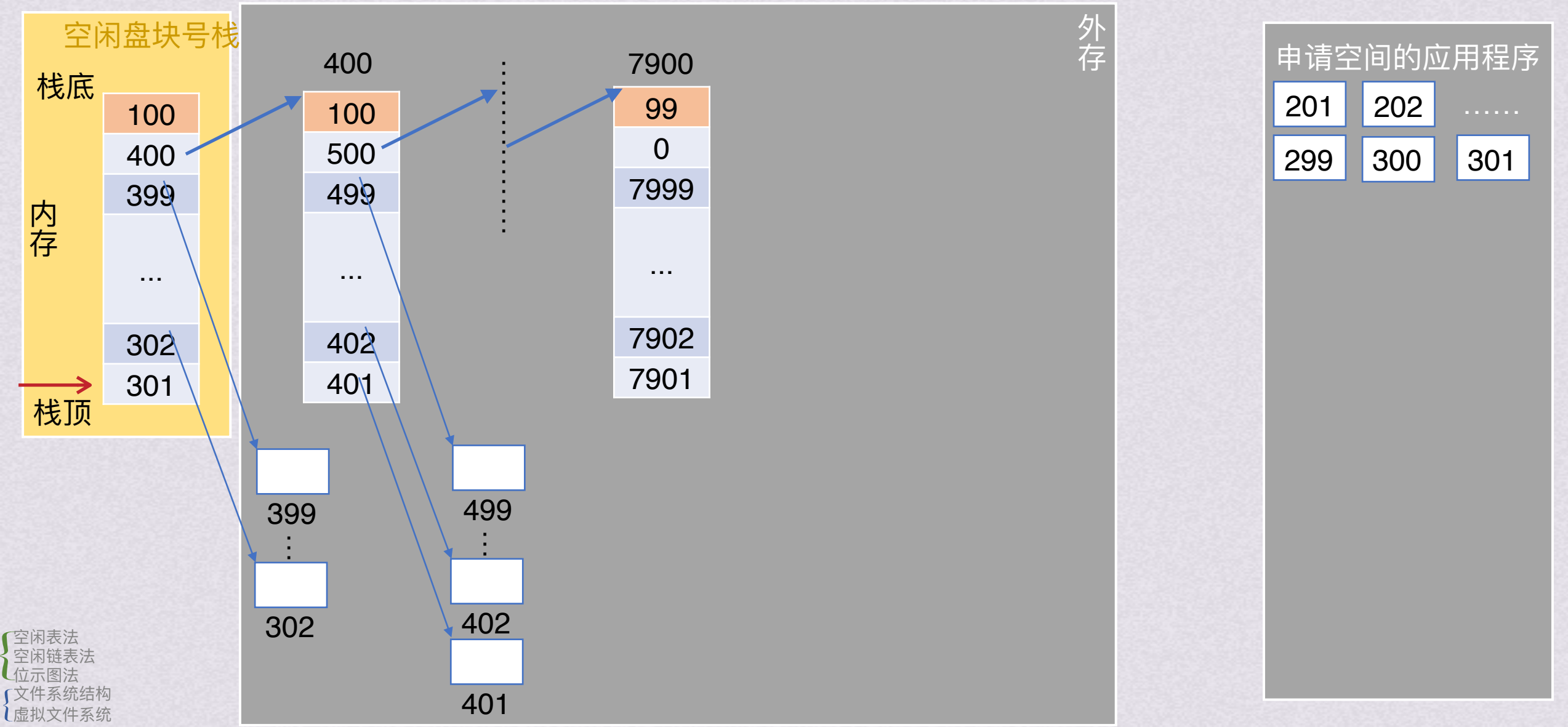
空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

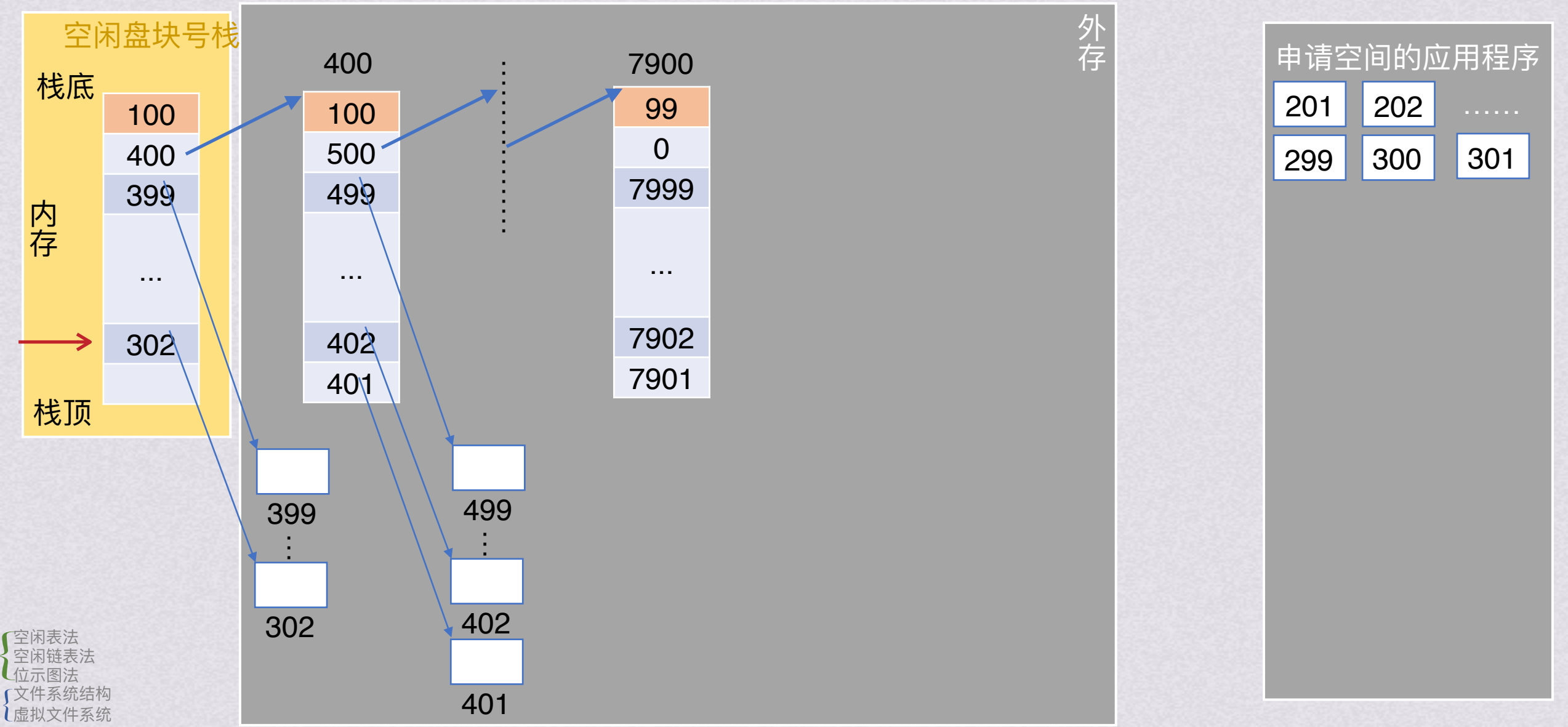


空闲表法
空闲链表法
位示图法
文件系统结构
虚拟文件系统

外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点

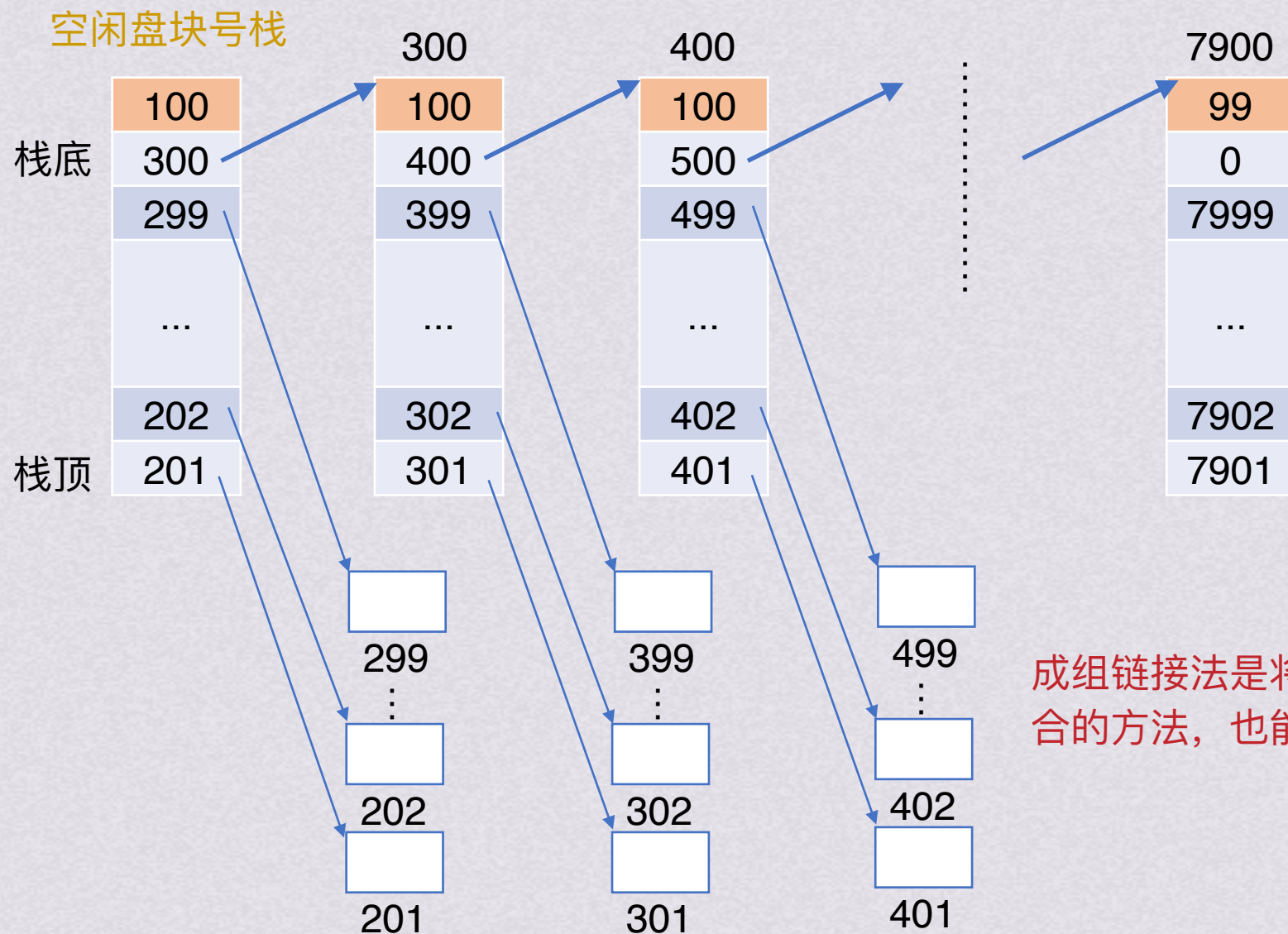




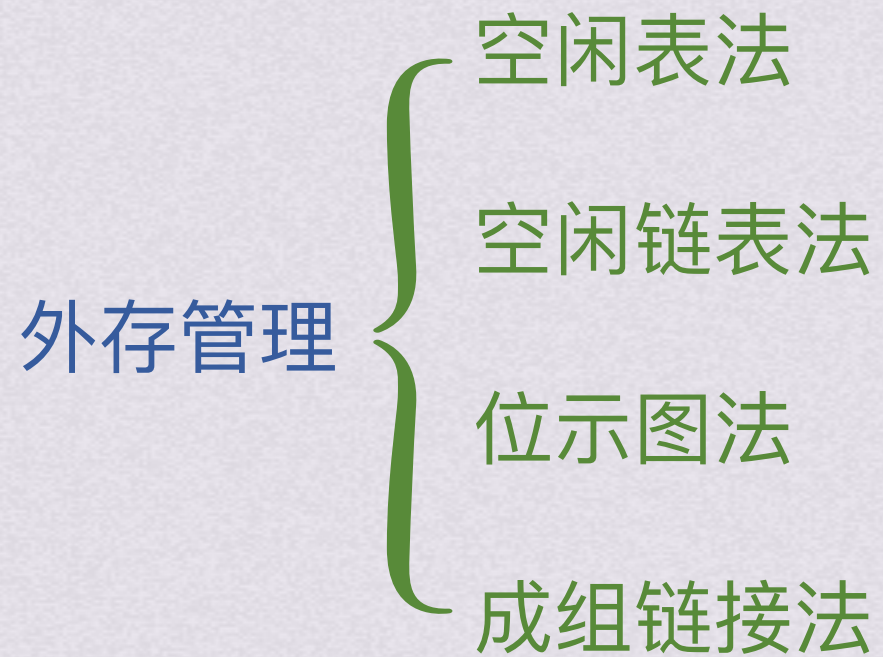
外存管理

成组链接法

空闲链表和空闲表法都不适合大型文件系统，这样会让空闲表或者空闲链表太长，UNIX中采用成组链接法克服这一缺点



成组链接法是将连续分配和离散分配相结合的方法，也能方便找到连续的空闲块



注：FAT也可以用于空闲空间管理

